Data Science sous R

Chapitre 4 — Représentations graphiques et régression

ESME — 2026-2027

Table des matières

4 Représentations graphiques et régression **2**
4.1 Pourquoi visualiser ? 2
4.2 Graphiques avec R de base 4
4.3 Graphiques avec `ggplot2` 5
4.4 Régression linéaire simple 19
4.5 Régression linéaire multiple 22
4.6 Régression logistique 24
4.7 Résumé 27
4.8 Exercices 27

Chapitre 4

Représentations graphiques et régression

Objectifs du chapitre

- Connaître les principes d'un bon graphique et choisir un graphique adapté aux données.
- Produire des graphiques avec R de base et, surtout, avec `ggplot2` : *geoms*, mappages, positions, *facets*, *scales*, thèmes.
- Ajuster, interpréter et critiquer une régression linéaire simple et multiple, puis une régression logistique.

Support de cours : document ***Graphiques avec ggplot2.pdf****. Les exercices sont regroupés en fin de chapitre.*

```
library(tidyverse)
library(questionr)   # jeu de données rp2018 (exercices)
library(ISLR)        # jeux de données Carseats et OJ
source("R/boutique.R")
commandes <- lire_boutique("commandes")
clients   <- lire_boutique("clients")
```

4.1 Pourquoi visualiser ?

La visualisation est un élément essentiel de l'analyse : elle révèle des structures que les résumés numériques cachent. Le **quartet d'Anscombe** (jeu `anscombe`) contient quatre séries (x, y) qui ont presque exactement la même moyenne, la même variance, la même corrélation et la même droite de régression :

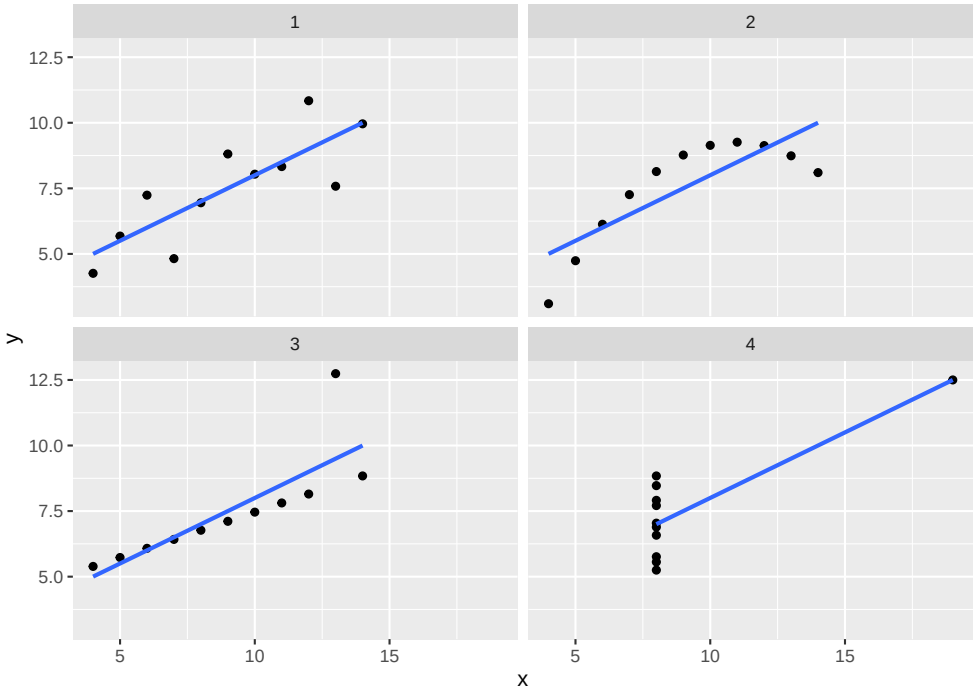
```
anscombe_long <- tibble(
  serie = rep(1:4, each = 11),
  x = c(anscombe$x1, anscombe$x2, anscombe$x3, anscombe$x4),
  y = c(anscombe$y1, anscombe$y2, anscombe$y3, anscombe$y4)
)
anscombe_long |>
  group_by(serie) |>
  summarise(moy_x = mean(x), moy_y = mean(y), var_y = var(y), correlation =
   ↪  cor(x, y))
```

```
#> # A tibble: 4 x 5
```

```
#>   serie moy_x moy_y var_y correlation
#>   <int> <dbl> <dbl> <dbl>       <dbl>
#> 1     1     9  7.50  4.13       0.816
#> 2     2     9  7.50  4.13       0.816
#> 3     3     9  7.5   4.12       0.816
#> 4     4     9  7.50  4.12       0.817
```

Pourtant, leurs graphiques n'ont rien à voir :

```
ggplot(anscombe_long, aes(x, y)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, formula = y ~ x) +
  facet_wrap(vars(serie))
```



Toujours regarder les données avant (et après) de les modéliser. Explorer graphiquement est un processus **itératif** : on commence simple, puis on affine en regardant les données sous différents angles.

4.1.1 Principes d'un bon graphique

Edward Tufte (*The Visual Display of Quantitative Information*) propose quelques principes :

- montrer les données avant tout ;
- ne pas les déformer (axes tronqués trompeurs, effets 3D) ;
- encourager l'œil à **comparer** ;
- montrer plusieurs niveaux de détail, de la vue d'ensemble à la structure fine.

Un graphique se décompose en quatre éléments : les **indices visuels** (position, longueur, couleur, taille…), le **système de coordonnées** (cartésien, polaire, géographique), les **échelles** (linéaire, logarithmique, catégorielle, temporelle) et le **contexte** (titre, étiquettes d'axes avec unités, source, lignes de référence).

L'œil compare très bien des **positions** et des **longueurs** sur une échelle commune, beaucoup moins bien des angles, des surfaces ou des couleurs. C'est pourquoi un diagramme en barres est presque toujours plus lisible qu'un diagramme circulaire (« camembert »).

4.1.2 Choisir un graphique

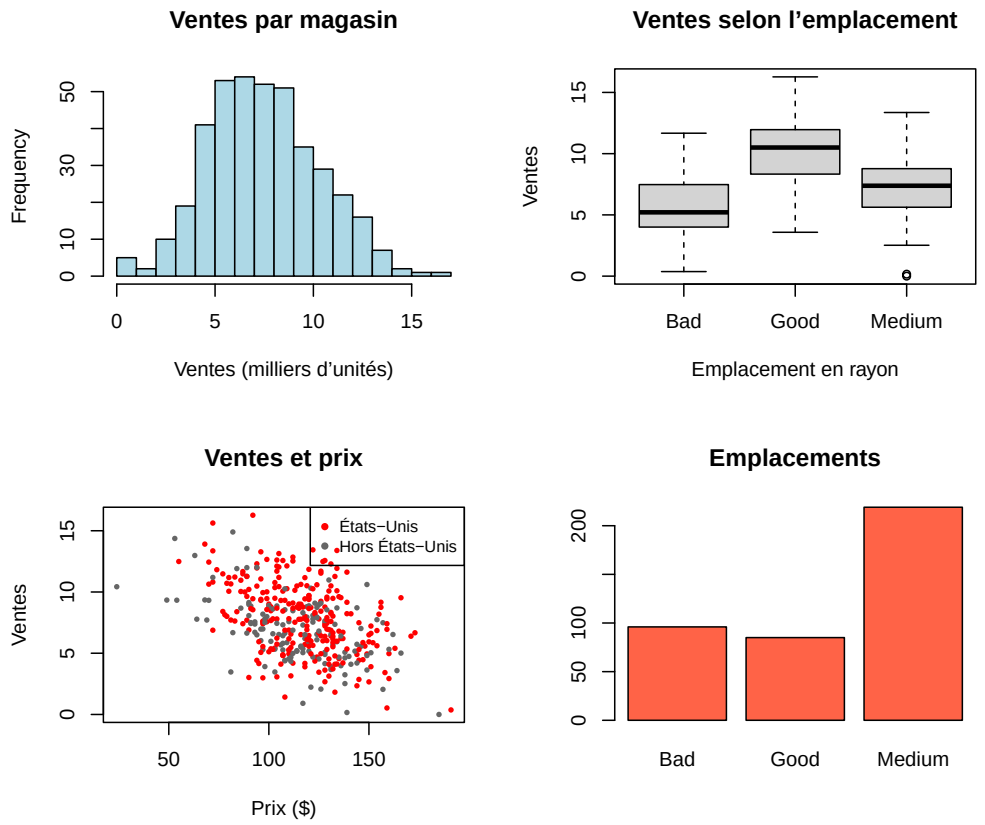
Question	Variables	Graphique	ggplot2
Distribution	1 quantitative	histogramme, densité, boîte à moustaches	`geom_histogram()`, `geom_density()`
Répartition	1 qualitative	diagramme en barres	`geom_bar()`, `geom_col()`
Relation	2 quantitatives	nuage de points	`geom_point()`
Comparaison	quantitative × qualitative	boîtes à moustaches, violons	`geom_boxplot()`, `geom_violin()`
Association	2 qualitatives	barres empilées ou juxtaposées	`geom_bar(position = ...)`
Évolution	quantitative × temps	courbe	`geom_line()`

4.2 Graphiques avec R de base

Les fonctions graphiques de base, vues au chapitre précédent, sont rapides pour explorer :

```
par(mfrow = c(2, 2))   # grille 2 x 2

hist(Carseats$Sales, breaks = 20, col = "lightblue",
     main = "Ventes par magasin", xlab = "Ventes (milliers d'unités)")
boxplot(Sales ~ ShelveLoc, data = Carseats,
        main = "Ventes selon l'emplacement", xlab = "Emplacement en rayon",
        ↪  ylab = "Ventes")
plot(Sales ~ Price, data = Carseats, pch = 16, cex = 0.6,
     col = ifelse(Carseats$US == "Yes", "red", "grey40"),
     main = "Ventes et prix", xlab = "Prix ($)", ylab = "Ventes")
legend("topright", legend = c("États-Unis", "Hors États-Unis"),
       col = c("red", "grey40"), pch = 16, cex = 0.8)
barplot(table(Carseats$ShelveLoc), main = "Emplacements", col = "tomato")
```




```
par(mfrow = c(1, 1))    # rétablir la grille par défaut
```

Paramètre	Rôle	Exemple
`main`, `xlab`, `ylab`	titres	`main = "Titre"`
`col`	couleur	`col = "steelblue"`
`pch`	symbole des points	`pch = 16` (disque plein)
`cex`	taille relative	`cex = 0.5`
`lty`, `lwd`	type et épaisseur de ligne	`lty = 2` (tirets)
`xlim`, `ylim`	bornes des axes	`xlim = c(0, 10)`

Les fonctions `abline()`, `lines()`, `points()`, `text()` et `legend()` **ajoutent** des éléments au graphique courant. Pour enregistrer : `png("fichier.png")`, puis le graphique, puis `dev.off()`, ou le bouton *Export* de l'onglet *Plots*.

4.3 Graphiques avec `ggplot2`

`ggplot2` repose sur une **grammaire des graphiques** : un graphique est construit en **superposant des couches** avec l'opérateur `+`. Une **esthétique** (*aesthetic*) est une correspondance explicite entre une **variable** et un **indice visuel**.

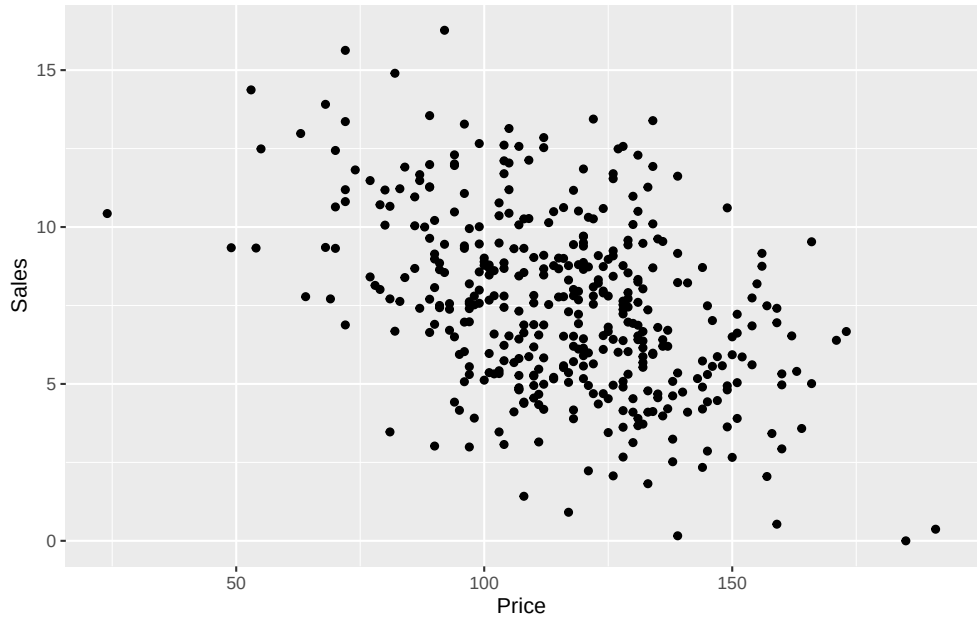
Composant	Rôle	Exemple
données	un tableau (format *tidy*)	`ggplot(Carseats)`
mappages (`aes()`)	lien entre variables et attributs graphiques	`aes(x = Price, y = Sales, color = ShelveLoc)`

Composant	Rôle	Exemple
geoms	forme géométrique	`geom_point()`, `geom_boxplot()`
facets	petits multiples	`facet_wrap(vars(ShelveLoc))`
scales	réglage des axes, couleurs, tailles	`scale_y_log10()`
thème et titres	apparence générale	`labs()`, `theme_minimal()`

4.3.1 Initialisation et premier *geom*

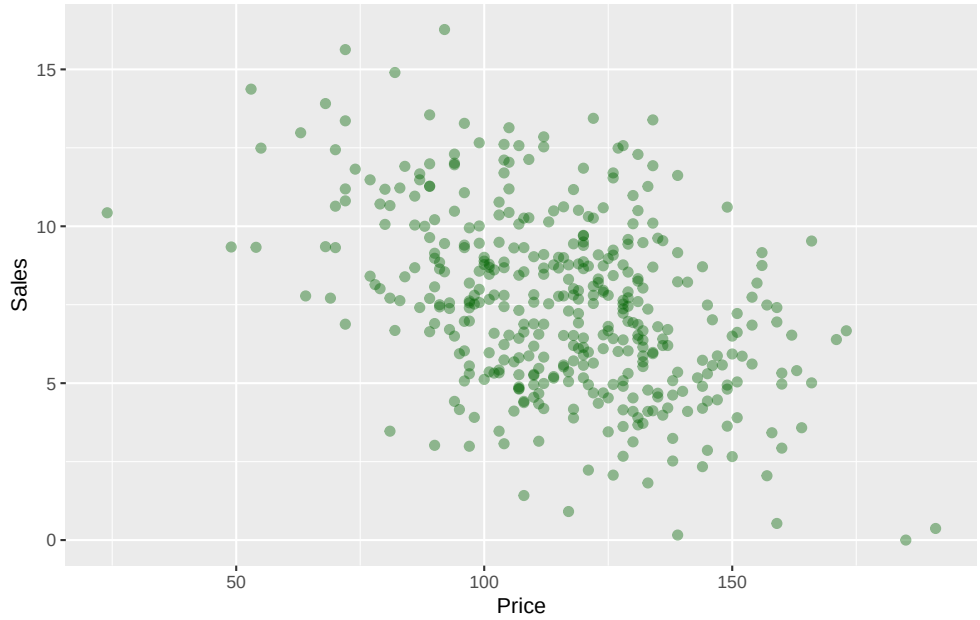
Un graphique s'initialise avec `ggplot()` et un tableau de données. On ajoute ensuite un *geom*, auquel on indique les variables à représenter via `aes()`. Les variables s'écrivent sans le nom du tableau (`Price` et non `Carseats$Price`).

```
ggplot(Carseats) +
  geom_point(aes(x = Price, y = Sales))
```



Les attributs **constants** se donnent dans le *geom*, **hors** de `aes()` :

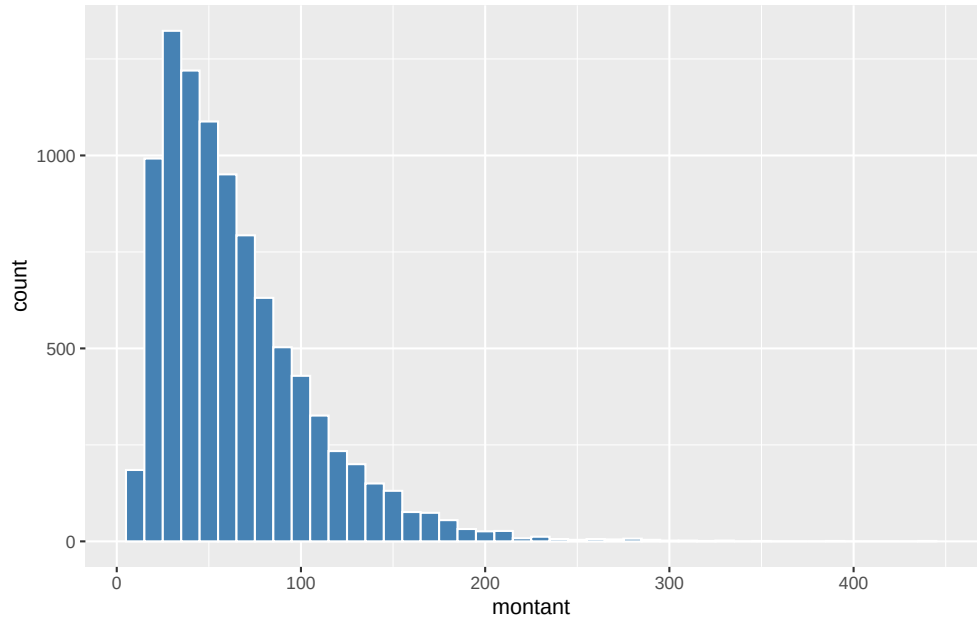
```
ggplot(Carseats) +
  geom_point(aes(x = Price, y = Sales), color = "darkgreen", size = 2, alpha =
  ↪  0.4)
```



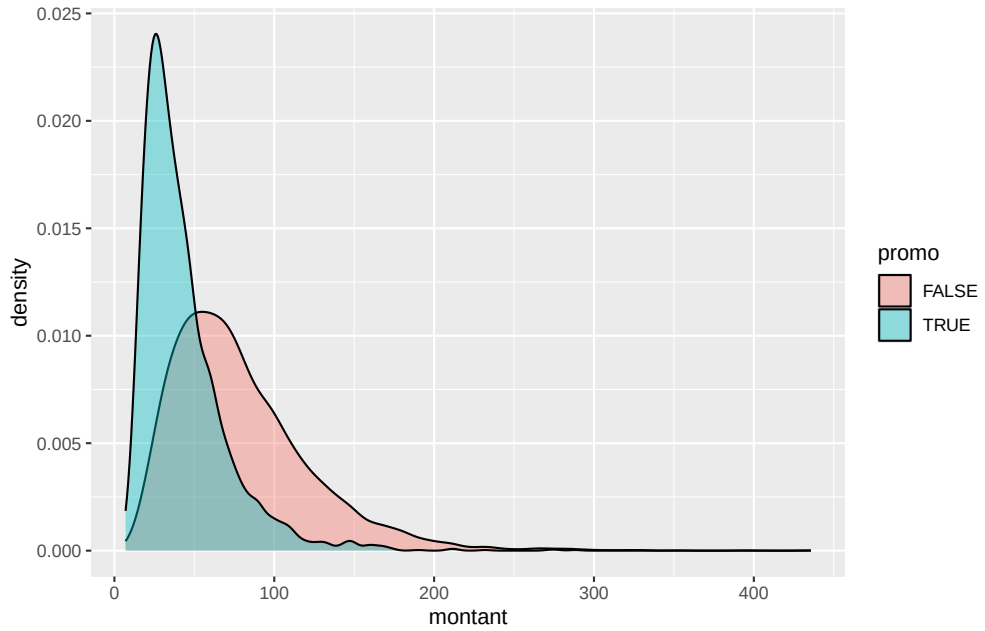
4.3.2 Les principaux *geoms*

Histogramme et densité — la distribution des montants de commande :

```
ggplot(commandes) +
  geom_histogram(aes(x = montant), binwidth = 10, fill = "steelblue", color =
   ↪  "white")
```




```
ggplot(commandes) +
  geom_density(aes(x = montant, fill = promo), alpha = 0.4)
```

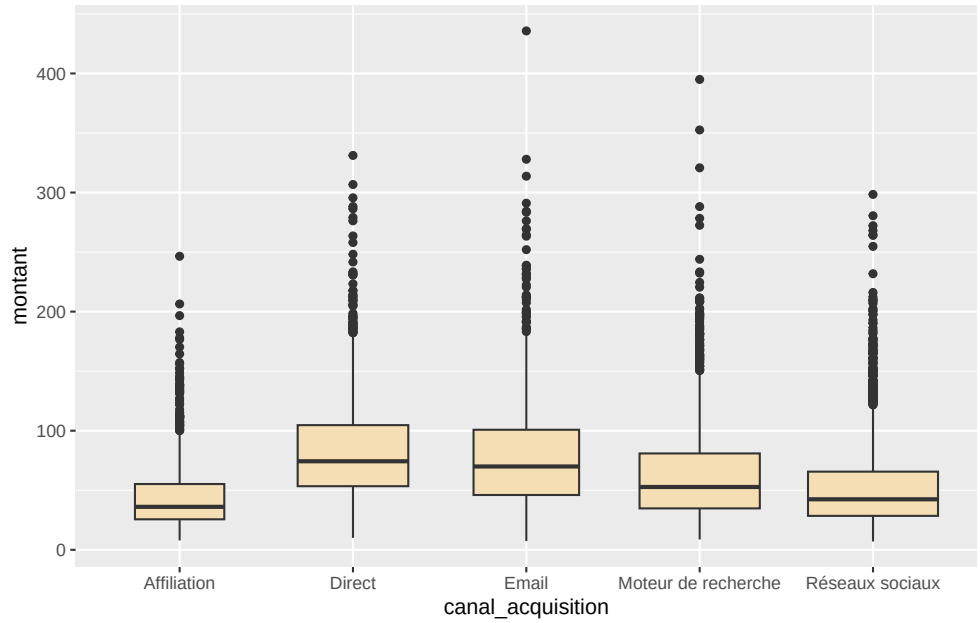


L'argument `binwidth` (ou `bins`) d'un histogramme et `bw` d'une densité règlent le niveau de détail : à tester, car ils peuvent masquer ou inventer des structures.

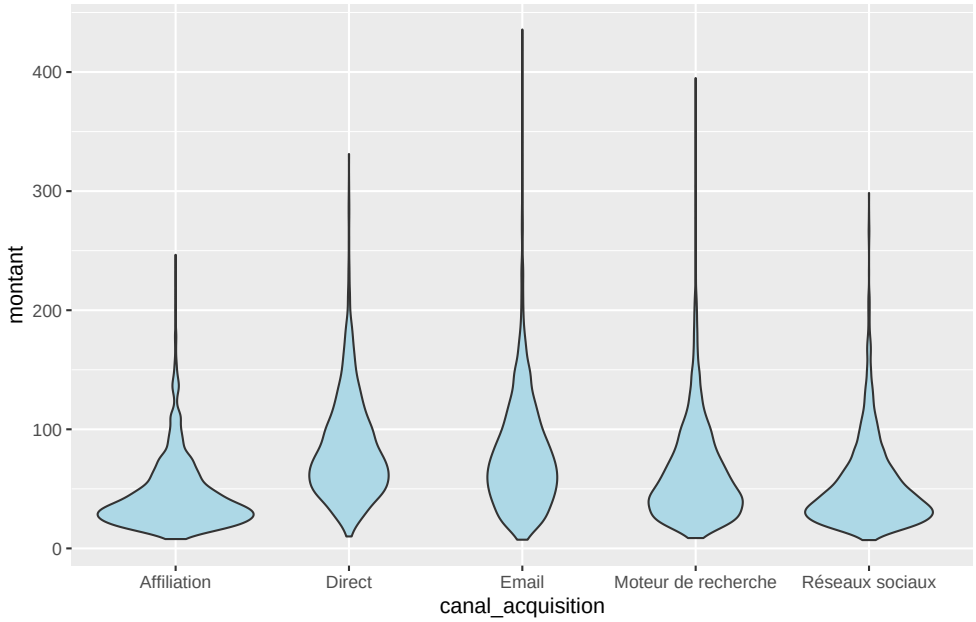
Boîtes à moustaches et violons — le panier selon le canal d'acquisition :

```
commandes_clients <- left_join(commandes, clients, by = "client_id")

ggplot(commandes_clients) +
  geom_boxplot(aes(x = canal_acquisition, y = montant), fill = "wheat",
   ↪  varwidth = TRUE)
```



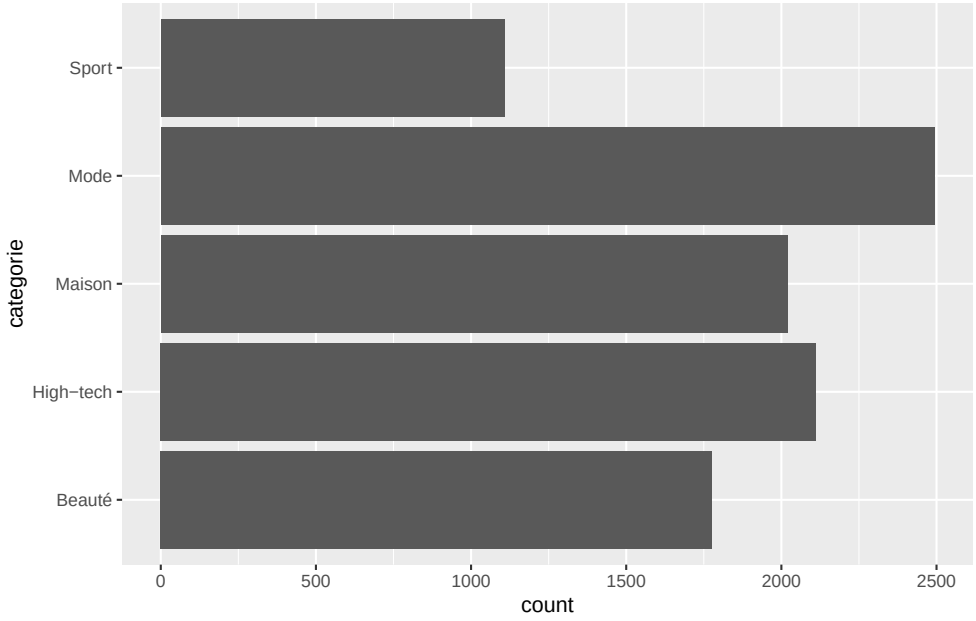

```
ggplot(commandes_clients) +
  geom_violin(aes(x = canal_acquisition, y = montant), fill = "lightblue")
```



`varwidth = TRUE` rend la largeur des boîtes proportionnelle aux effectifs ; le violon montre la forme complète de la distribution.

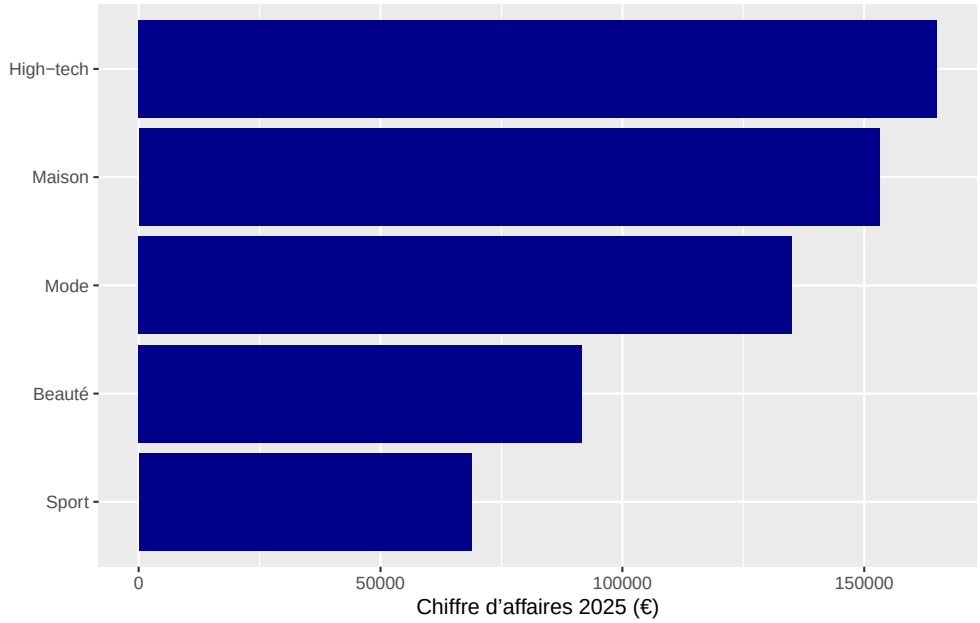
Barres : `geom_bar()` et `geom_col()` — `geom_bar()` **compte** les lignes de chaque modalité ; `geom_col()` représente des valeurs **déjà calculées**. Passer la variable en `y` donne des barres horizontales, plus lisibles avec des étiquettes longues (on peut aussi ajouter `coord_flip()`).

```
ggplot(commandes) +
  geom_bar(aes(y = categorie))
```




```
ca_categorie <- commandes |>
  group_by(categorie) |>
  summarise(ca = sum(montant))

ggplot(ca_categorie) +
  geom_col(aes(x = ca, y = reorder(categorie, ca)), fill = "darkblue") +
  labs(x = "Chiffre d'affaires 2025 (€)", y = NULL)
```

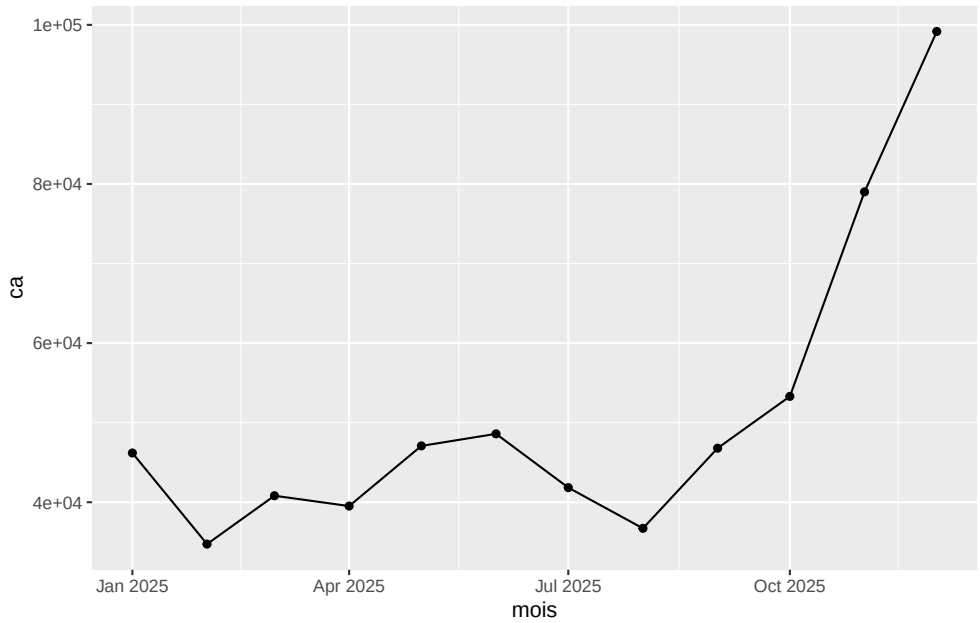


`reorder(categorie, ca)` trie les barres selon le chiffre d'affaires : un tri rend la lecture beaucoup plus rapide.

Courbes — l'évolution du chiffre d'affaires mensuel :

```
ca_mensuel <- commandes |>
  mutate(mois = floor_date(date, "month")) |>
  group_by(mois) |>
  summarise(ca = sum(montant))

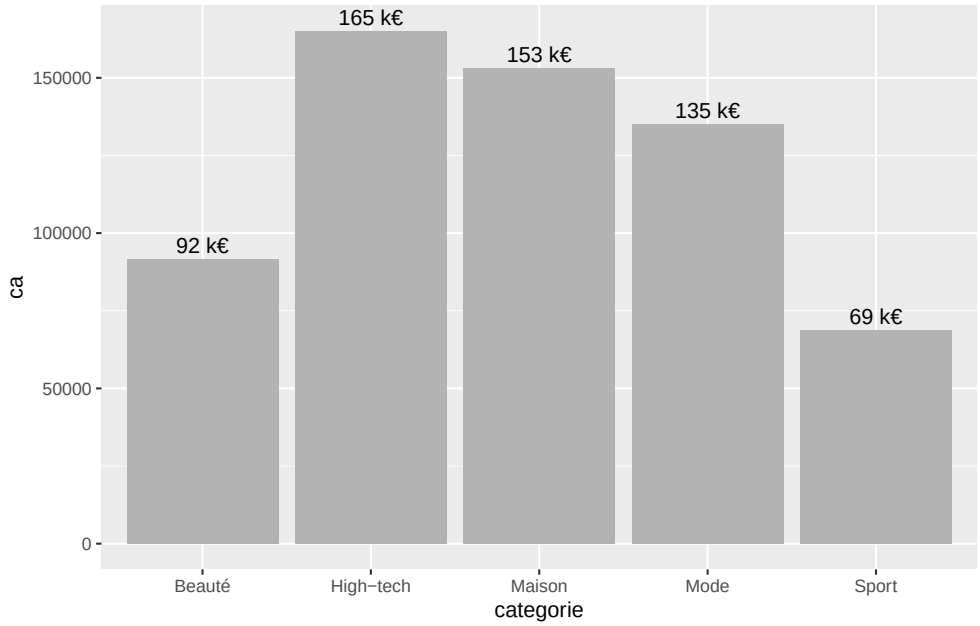
ggplot(ca_mensuel, aes(x = mois, y = ca)) +
  geom_line() +
  geom_point()
```



La saisonnalité est nette : les ventes s'envolent en novembre (*Black Friday*) et en décembre (Noël).

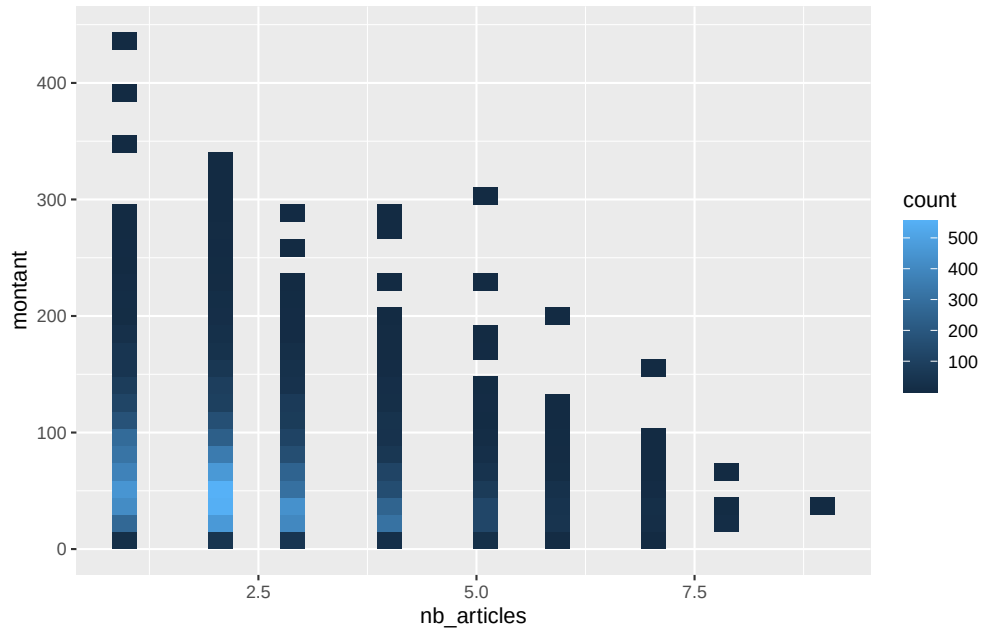
Texte et étiquettes — `geom_text()` et `geom_label()` demandent `x`, `y` et `label` :

```r
ggplot(ca_categorie, aes(x = categorie, y = ca)) +
  geom_col(fill = "grey70") +
  geom_text(aes(label = paste(round(ca / 1000), "k€")), vjust = -0.4)
```



Nombreux points — quand les points se superposent, `geom_bin2d()` (ou `geom_hex()`, qui nécessite le package `hexbin`) colore une grille selon le nombre de points :

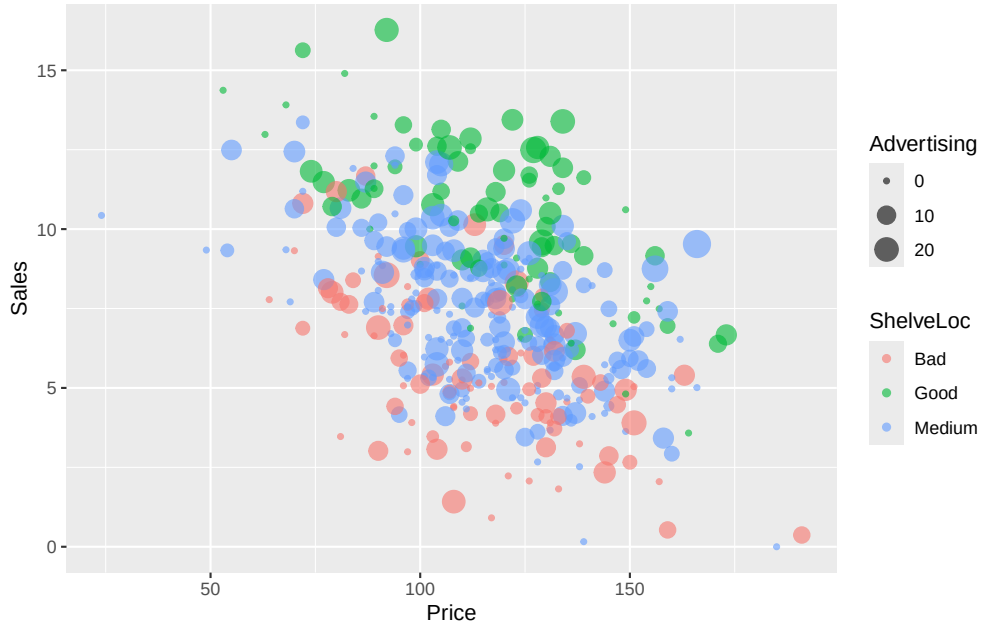
```r
ggplot(commandes) +
  geom_bin2d(aes(x = nb_articles, y = montant), bins = 30)
```



4.3.3 Mappages

Un **mappage** relie un attribut graphique (position, couleur, taille, transparence, forme) à une **variable**. On peut en combiner plusieurs :

```r
ggplot(Carseats) +
  geom_point(aes(x = Price, y = Sales, color = ShelveLoc, size = Advertising),
    ↪  alpha = 0.6)
```



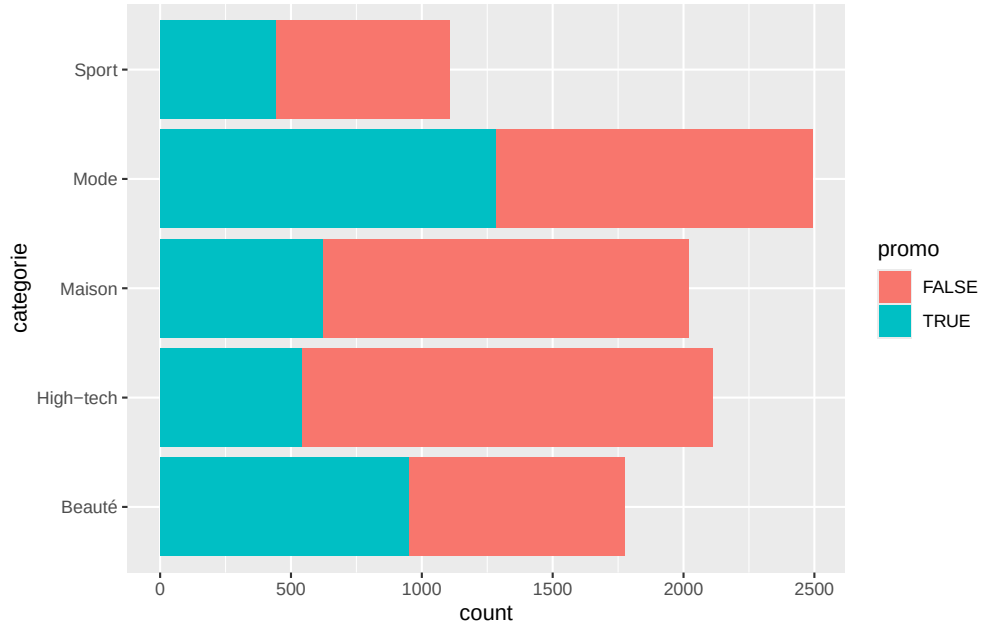
Ce graphique montre quatre variables : à prix égal, un magasin où le produit est **bien placé en rayon** (`Good`) vend nettement plus.

> **`aes()` ou pas `aes()` ?** Si l'attribut dépend d'une **variable**, c'est un mappage : il va **dans** `aes()`. S'il est **constant**, il va **hors** de `aes()`. Écrire `aes(color = "blue")` crée une légende « blue » et une couleur imprévue.

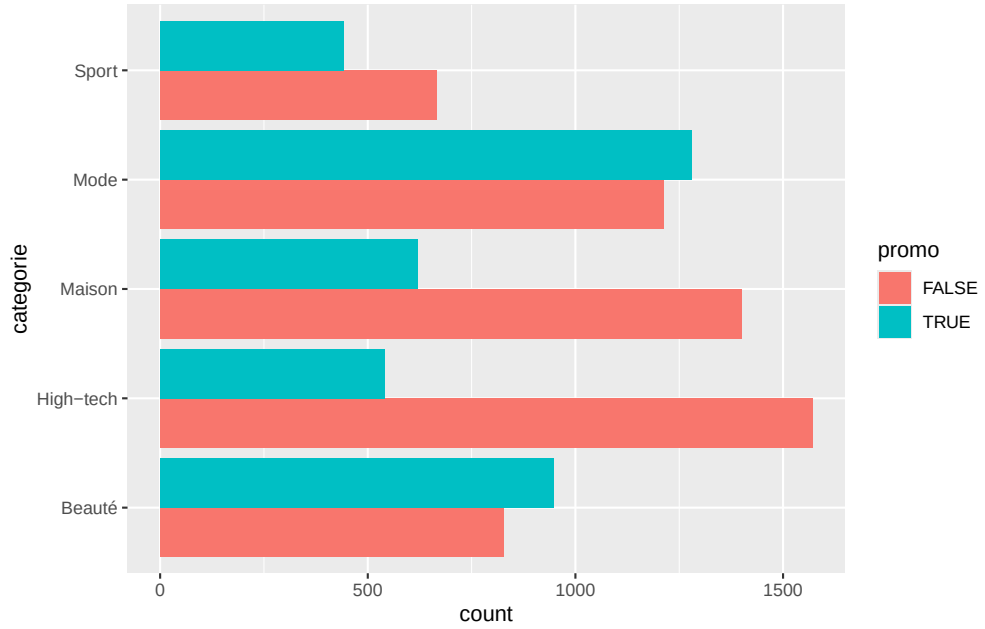
4.3.4 Positions des barres

Avec un mappage `fill`, l'argument `position` de `geom_bar()` contrôle l'agencement des barres :

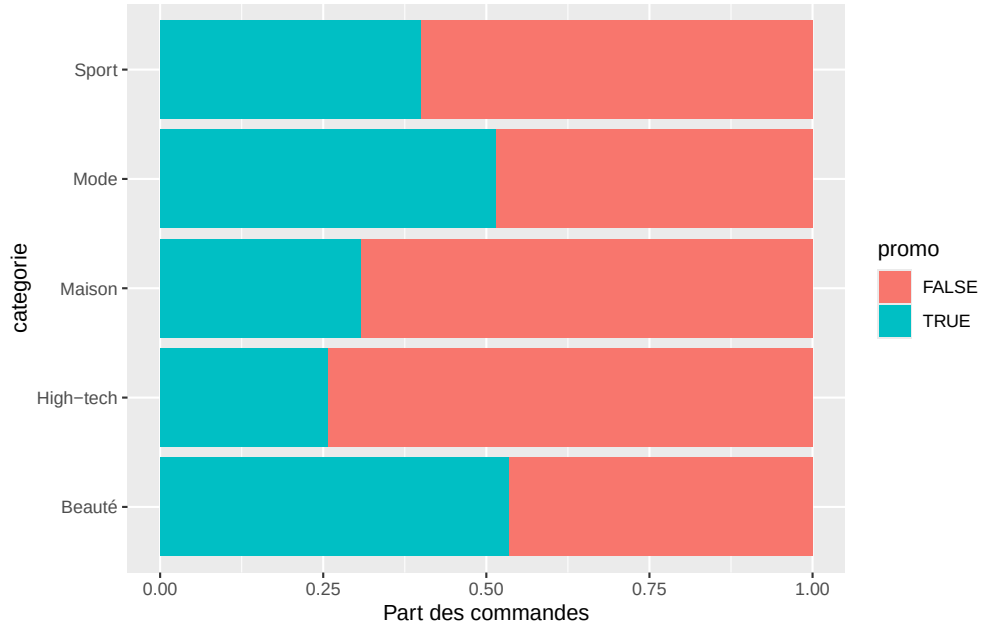
```r
ggplot(commandes) +
  geom_bar(aes(y = categorie, fill = promo))                    # empilées
    ↪  (défaut)
```




```
ggplot(commandes) +
  geom_bar(aes(y = categorie, fill = promo), position = "dodge")  # côte à côte
```




```
ggplot(commandes) +
  geom_bar(aes(y = categorie, fill = promo), position = "fill") + # proportions
  labs(x = "Part des commandes")
```

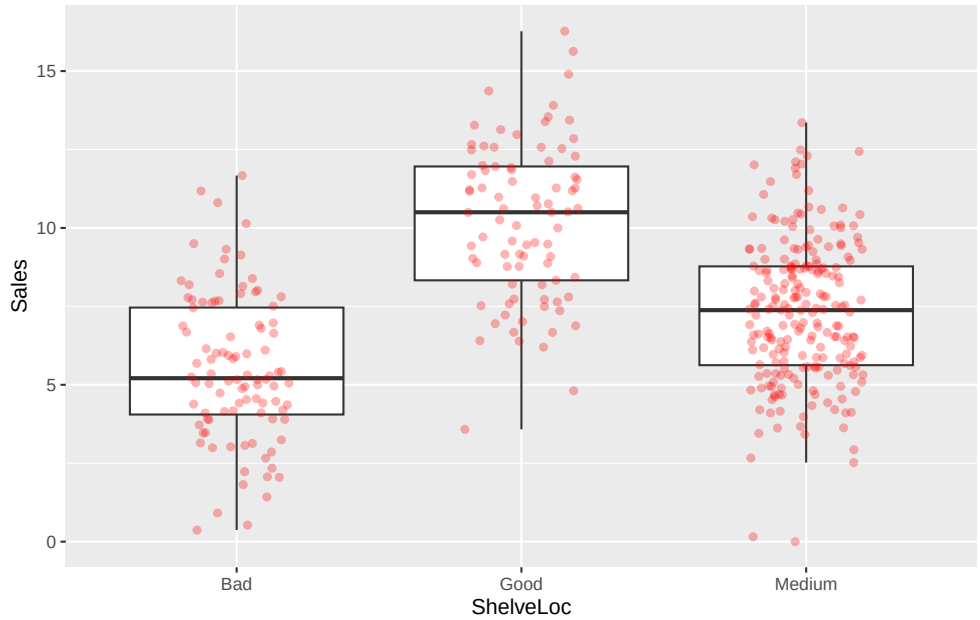


Les promotions sont surtout utilisées en Mode et en Beauté. Pour comparer des **proportions** entre groupes de tailles différentes, `position = "fill"` est le bon choix.

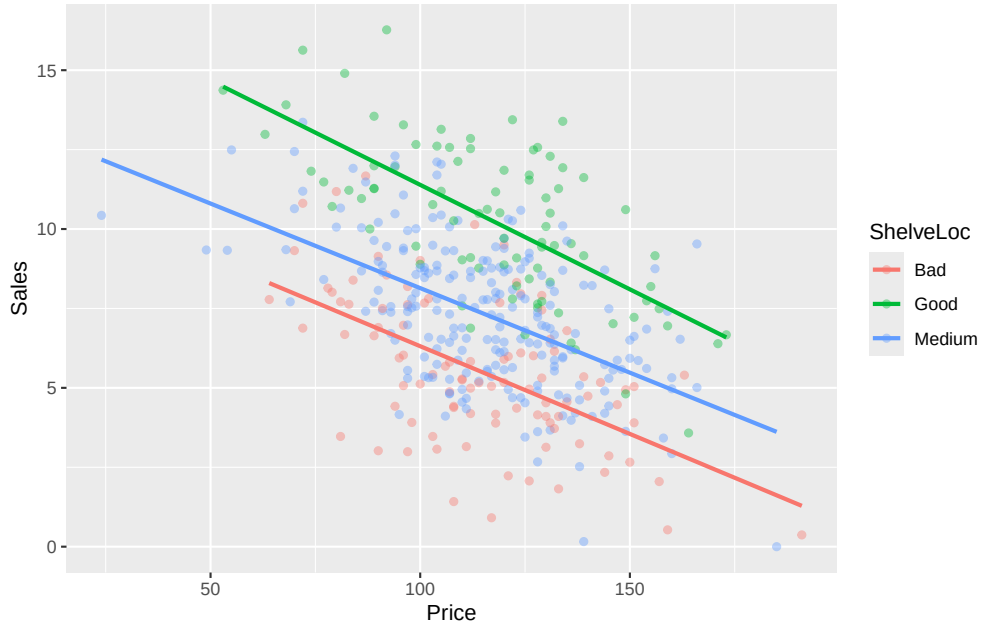
4.3.5 Superposer plusieurs *geoms*

On ajoute les couches les unes après les autres. Les mappages déclarés dans `ggplot()` sont **hérités** par tous les *geoms* :

```
ggplot(Carseats, aes(x = ShelveLoc, y = Sales)) +
  geom_boxplot(outlier.shape = NA) +
  geom_jitter(width = 0.2, alpha = 0.3, color = "red")
```




```
ggplot(Carseats, aes(x = Price, y = Sales, color = ShelveLoc)) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE)
```

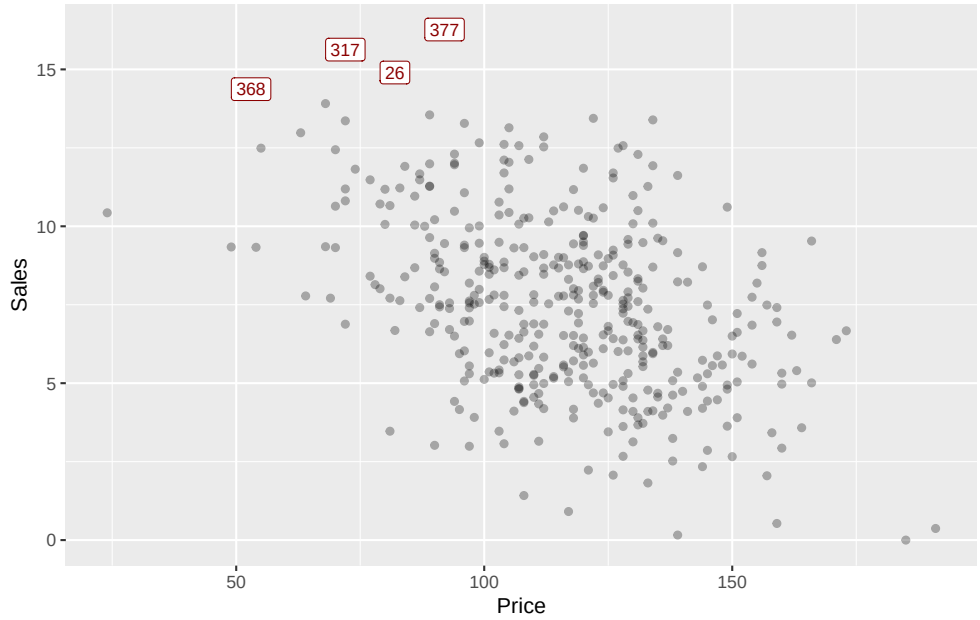


Les trois droites sont presque parallèles : l'effet du prix est à peu près le même quel que soit l'emplacement. Nous le vérifierons avec un modèle.

Chaque *geom* peut aussi avoir ses **propres données**, par exemple pour étiqueter seulement quelques points :

```
meilleurs <- Carseats |>
  mutate(magasin = row_number()) |>
  filter(Sales > 14)

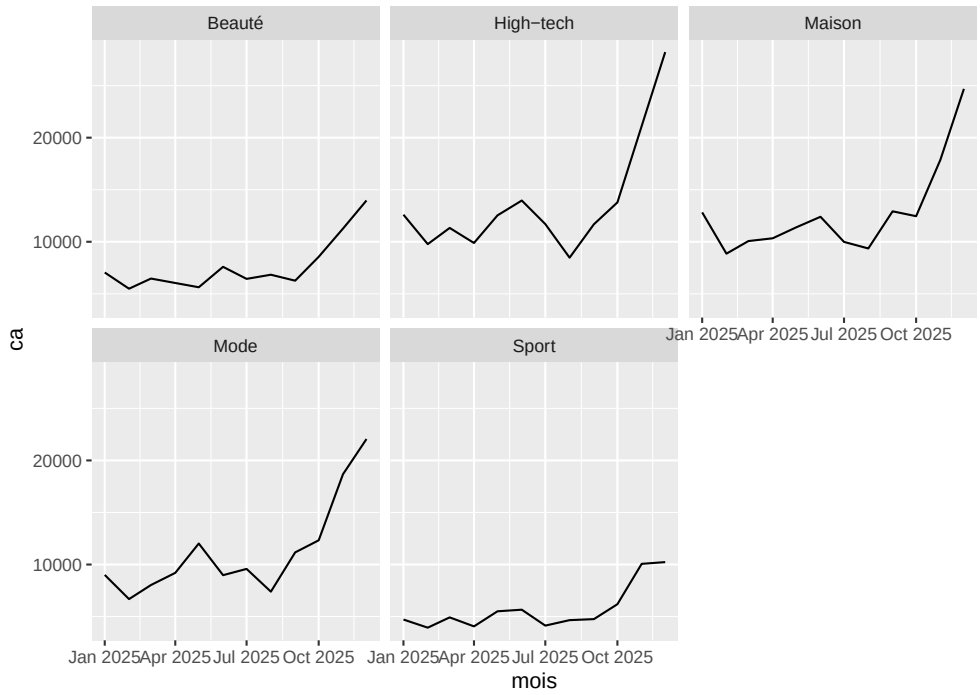
ggplot(Carseats, aes(x = Price, y = Sales)) +
  geom_point(alpha = 0.3) +
  geom_label(data = meilleurs, aes(label = magasin), color = "darkred", size =
   ↪  3)
```



4.3.6 *Faceting*

Le *faceting* répète le même graphique pour chaque modalité d'une variable qualitative, **avec les mêmes échelles**, ce qui permet la comparaison directe.

```
commandes |>
  mutate(mois = floor_date(date, "month")) |>
  group_by(categorie, mois) |>
  summarise(ca = sum(montant), .groups = "drop") |>
  ggplot(aes(x = mois, y = ca)) +
  geom_line() +
  facet_wrap(vars(categorie))
```



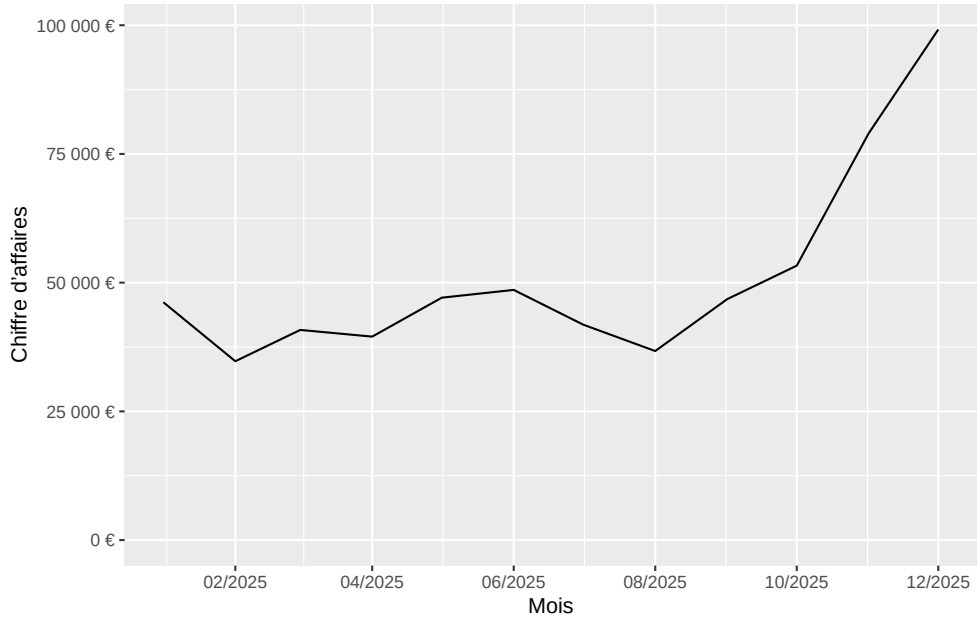
`facet_grid(rows = vars(a), cols = vars(b))` dispose les graphiques selon une grille croisant deux variables.

4.3.7 *Scales*

Les *scales* contrôlent la façon dont les valeurs sont traduites en attributs graphiques, et la légende ou l'axe correspondant. Elles prennent la forme `scale_<attribut>_<type>()`.

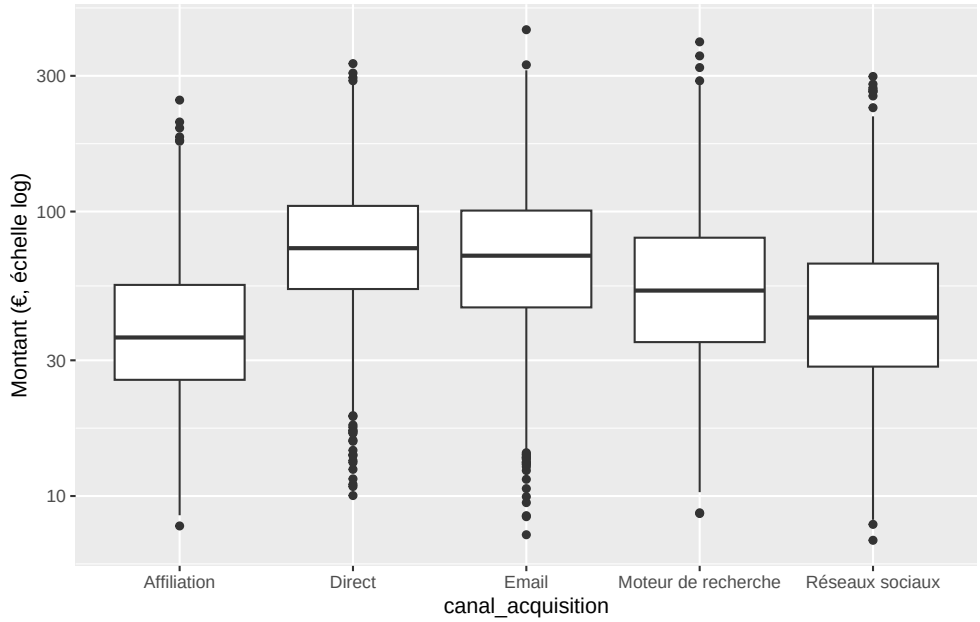
Axes — titres, limites, format des nombres, échelle logarithmique :

```
ggplot(ca_mensuel, aes(x = mois, y = ca)) +
  geom_line() +
  scale_x_date("Mois", date_labels = "%m/%Y", date_breaks = "2 months") +
  scale_y_continuous("Chiffre d'affaires", limits = c(0, NA),
                     labels = scales::label_number(suffix = " €", big.mark = "
                      ↪  "))
```



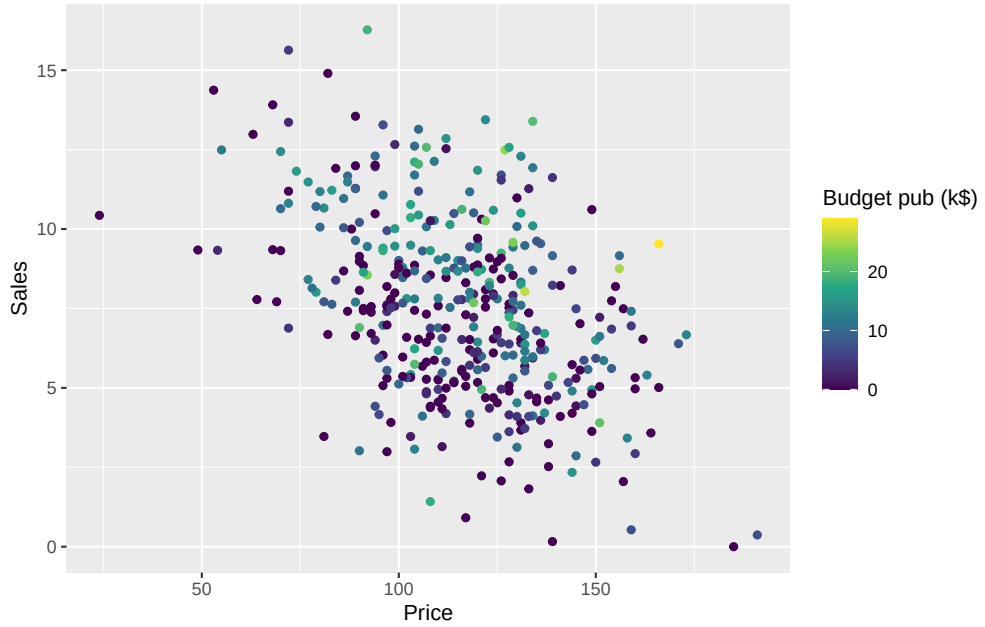
Faire partir l'axe des ordonnées de 0 évite d'exagérer visuellement les variations, ce qui est essentiel pour des barres. Les montants, très asymétriques, se lisent mieux en **échelle logarithmique** :

```
ggplot(commandes_clients) +
  geom_boxplot(aes(x = canal_acquisition, y = montant)) +
  scale_y_log10("Montant (€, échelle log)")
```

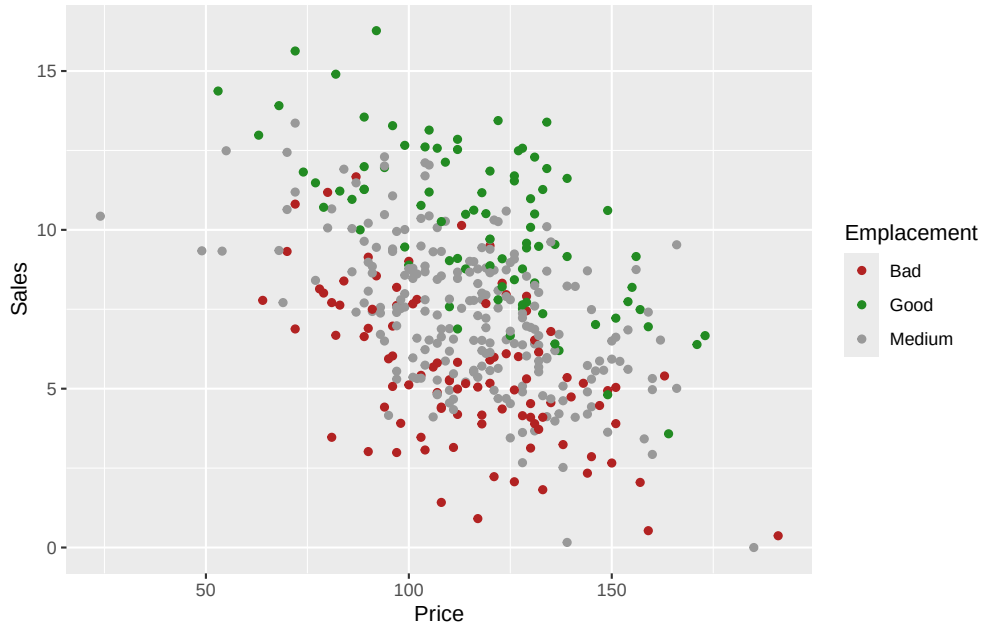


Couleurs — pour une variable quantitative, `scale_color_gradient()`, `scale_color_viridis_c()` ou `scale_color_distiller()` ; pour une variable qualitative, `scale_color_brewer()`, `scale_color_viridis_d()` ou `scale_color_manual()` :

```
ggplot(Carseats) +
  geom_point(aes(x = Price, y = Sales, color = Advertising)) +
  scale_color_viridis_c("Budget pub (k$)")
```




```
ggplot(Carseats) +
  geom_point(aes(x = Price, y = Sales, color = ShelveLoc)) +
  scale_color_manual("Emplacement",
                     values = c(Bad = "firebrick", Medium = "grey60", Good =
                      ↪  "forestgreen"))
```



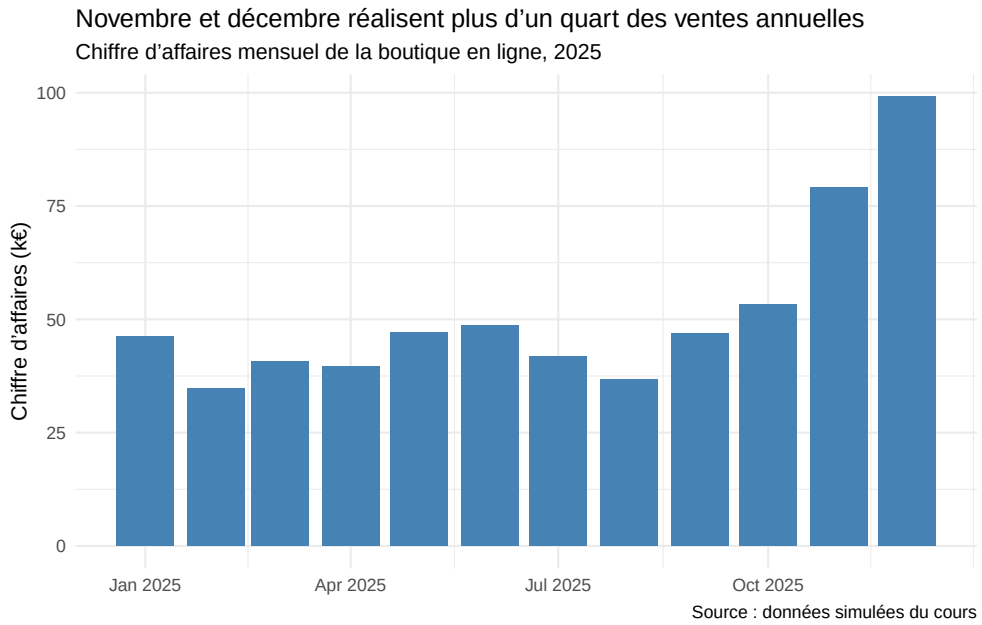
Les palettes `viridis` et certaines palettes ColorBrewer restent lisibles pour les personnes daltoniennes (`RColorBrewer::display.brewer.all()` les liste).

Tailles — `scale_size(range = c(min, max))`.

4.3.8 Titres et thèmes

`labs()` définit le titre, le sous-titre, la source et les noms des axes et légendes ; les thèmes (`theme_minimal()`, `theme_bw()`, `theme_classic()`…) règlent l'apparence générale, et `theme()` permet de modifier chaque élément.

```r
ggplot(ca_mensuel, aes(x = mois, y = ca / 1000)) +
  geom_col(fill = "steelblue") +
  labs(title = "Novembre et décembre réalisent plus d'un quart des ventes
   ↪  annuelles",
       subtitle = "Chiffre d'affaires mensuel de la boutique en ligne, 2025",
       x = NULL, y = "Chiffre d'affaires (k€)",
       caption = "Source : données simulées du cours") +
  theme_minimal()
```



Un graphique de présentation : un titre qui énonce le **message**, des axes avec unités, une source, des couleurs lisibles, aucun effet inutile. Enregistrer avec `ggsave("figure.png", width = 8, height = 5, dpi = 300)`.

Le package `esquisse` (menu *Addins* de RStudio, ou `esquisse::esquisser()`) permet de construire un graphique `ggplot2` à la souris, puis de récupérer le code correspondant : pratique pour explorer ou retrouver une syntaxe.

4.4 Régression linéaire simple

4.4.1 Le modèle

On cherche à expliquer une variable quantitative Y (la **réponse**) par une variable quantitative X (le **prédicteur**) :

$$Y_i = \beta_0 + \beta_1 X_i + \varepsilon_i, \qquad i = 1, \dots, n$$

- β_0 : ordonnée à l'origine ;
- β_1 : pente, c'est-à-dire la variation moyenne de Y quand X augmente d'une unité ;
- ε_i : erreur, supposée centrée, de variance constante σ^2, indépendante d'une observation à l'autre (et gaussienne pour les tests).

La méthode des **moindres carrés ordinaires** choisit $\hat\beta_0$ et $\hat\beta_1$ qui minimisent la somme des carrés des résidus $\sum_i (Y_i - \hat\beta_0 - \hat\beta_1 X_i)^2$. On obtient :

$$\hat{\beta}_1 = \frac{\text{cov}(X, Y)}{\text{var}(X)}, \qquad \hat{\beta}_0 = \bar{Y} - \hat{\beta}_1 \bar{X}$$

4.4.2 Ajuster le modèle avec `lm()`

Quel est le lien entre le prix pratiqué et les ventes ? La **formule** `Sales ~ Price` se lit « `Sales` expliqué par `Price` ». On passe le tableau avec l'argument `data` (plutôt que d'écrire `Carseats$Sales ~ Carseats$Price`, ce qui empêche ensuite d'utiliser `predict()`).

```
modele_1 <- lm(Sales ~ Price, data = Carseats)
summary(modele_1)
```

```
#>
#> Call:
#> lm(formula = Sales ~ Price, data = Carseats)
#>
#> Residuals:
#>    Min     1Q Median     3Q    Max
#> -6.522 -1.844 -0.146  1.650  7.511
#>
#> Coefficients:
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept) 13.64192    0.63281   21.56   <2e-16 ***
#> Price       -0.05307    0.00535   -9.91   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 2.53 on 398 degrees of freedom
#> Multiple R-squared:  0.198,  Adjusted R-squared:  0.196
#> F-statistic: 98.2 on 1 and 398 DF,  p-value: <2e-16
```

Lire la sortie de `summary()`

- *Estimate* : $\hat{\beta}_1 \approx$ -0.053 ; chaque dollar supplémentaire est associé, en moyenne, à 53 unités vendues en moins.
- *Std. Error* : précision de l'estimation.
- *t value* et *Pr(>|t|)* : test de $H_0 : \beta_1 = 0$. Une p-valeur très petite indique que la relation observée serait très improbable si la pente réelle était nulle.
- *Multiple R-squared* (R^2) : part de la variance des ventes expliquée par le modèle, ici environ 20 % ; le prix seul n'explique qu'une petite partie des différences entre magasins.
- *Residual standard error* : estimation de σ, l'erreur typique de prédiction (en milliers d'unités).

```
coef(modele_1)
```

```
#> (Intercept)       Price
#>    13.64192    -0.05307
```

```
confint(modele_1)        # intervalles de confiance à 95 %
```

```
#>                2.5 %   97.5 %
#> (Intercept) 12.3978 14.88599
#> Price       -0.0636 -0.04255
```

4.4.3 Prédire

```
scenarios <- tibble(Price = c(100, 120, 140))
predict(modele_1, newdata = scenarios, interval = "confidence")   # ventes
 ↪  moyennes
```

```
#>     fit   lwr   upr
#> 1 8.335 8.035 8.634
#> 2 7.273 7.020 7.526
#> 3 6.212 5.855 6.568
```

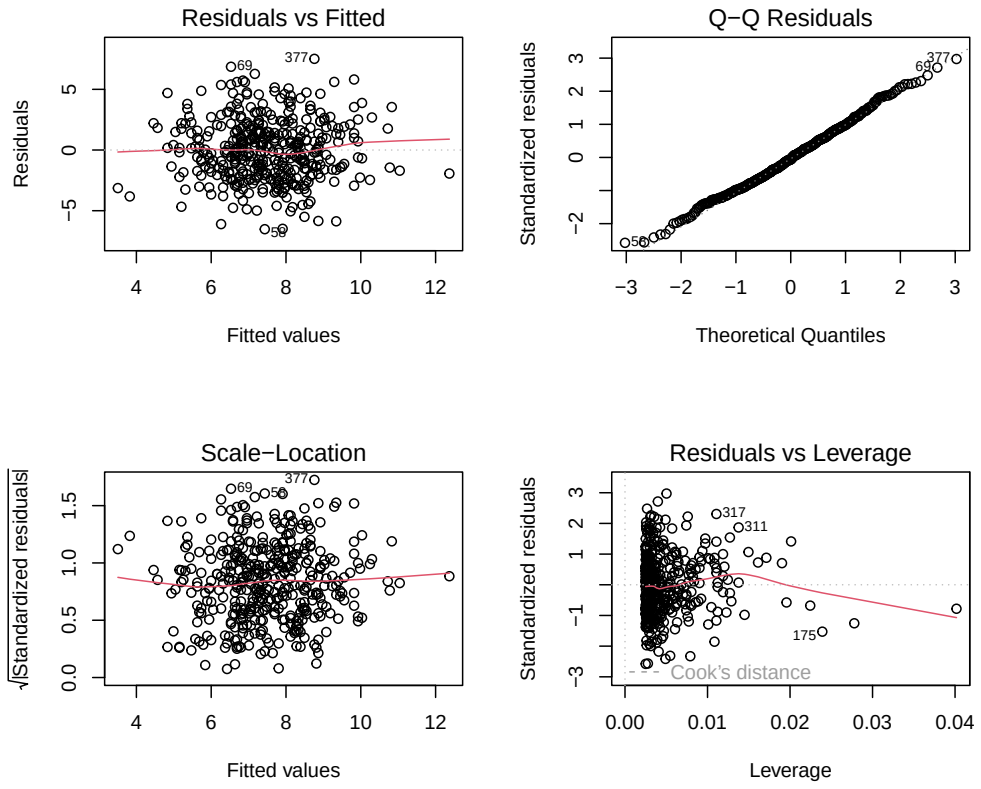
```
predict(modele_1, newdata = scenarios, interval = "prediction")   # pour un
 ↪  magasin
```

```
#>     fit   lwr   upr
#> 1 8.335 3.347 13.32
#> 2 7.273 2.288 12.26
#> 3 6.212 1.221 11.20
```

L'intervalle de **prédiction** (pour un magasin) est bien plus large que l'intervalle de **confiance** (pour la moyenne des magasins) : il inclut la variabilité d'un magasin à l'autre.

4.4.4 Vérifier les hypothèses : les résidus

```
par(mfrow = c(2, 2))
plot(modele_1)
```




```
par(mfrow = c(1, 1))
```

Graphique	Ce qu'on vérifie	Signe d'alerte
Residuals vs Fitted	linéarité	une courbure
Q-Q Residuals	normalité des résidus	points loin de la diagonale
Scale-Location	variance constante	un entonnoir
Residuals vs Leverage	points influents	points au-delà des courbes de Cook

Corrélation n'est pas causalité. Ces données sont **observationnelles** : les magasins n'ont pas reçu leur prix au hasard. Un magasin peut vendre cher *parce que* sa clientèle est aisée, ou parce que le concurrent est cher aussi. Pour mesurer l'effet causal d'un prix, l'idéal est une **expérience** (test A/B de prix). À défaut, il faut au moins tenir compte des autres facteurs : c'est le rôle de la régression multiple.

4.5 Régression linéaire multiple

4.5.1 Le modèle

$$Y_i = \beta_0 + \beta_1 X_{i1} + \cdots + \beta_p X_{ip} + \varepsilon_i$$

Chaque β_j s'interprète **toutes les autres variables étant fixées** (*ceteris paribus*).

```
modele_2 <- lm(Sales ~ Price + CompPrice + Advertising + ShelveLoc + Age +
 ↪  Income,
               data = Carseats)
summary(modele_2)
```

```
#>
#> Call:
#> lm(formula = Sales ~ Price + CompPrice + Advertising + ShelveLoc +
#>     Age + Income, data = Carseats)
#>
#> Residuals:
#>    Min     1Q Median     3Q    Max
#> -2.773 -0.695  0.028  0.673  3.329
#>
#> Coefficients:
#>                 Estimate Std. Error t value Pr(>|t|)
#> (Intercept)      5.47523    0.50501   10.84   <2e-16 ***
#> Price           -0.09532    0.00267  -35.70   <2e-16 ***
#> CompPrice        0.09257    0.00412   22.45   <2e-16 ***
#> Advertising      0.11590    0.00772   15.01   <2e-16 ***
#> ShelveLocGood    4.83567    0.15250   31.71   <2e-16 ***
#> ShelveLocMedium  1.95199    0.12538   15.57   <2e-16 ***
#> Age             -0.04613    0.00318  -14.52   <2e-16 ***
#> Income           0.01578    0.00184    8.59   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 1.02 on 392 degrees of freedom
#> Multiple R-squared:  0.872,  Adjusted R-squared:  0.87
#> F-statistic:  381 on 7 and 392 DF,  p-value: <2e-16
```

Interprétation marketing

- `Price` : à prix du concurrent et autres facteurs égaux, chaque dollar de plus fait perdre environ 95 unités. L'effet est presque **deux fois plus fort** que dans le modèle simple : dans les données, les magasins chers sont souvent face à des concurrents chers (`CompPrice`), ce qui masquait une partie de l'effet du prix. Oublier une variable importante **biaise** les coefficients des autres.
- `CompPrice` : quand le concurrent augmente son prix d'un dollar, nos ventes augmentent d'environ 93 unités.
- `Advertising` : 1 000 $ de publicité locale en plus sont associés à environ 116 unités vendues en plus. Si chaque unité rapporte une marge de 5 $, la publicité ne serait pas rentable à la marge ($116 \times 5\ \$ < 1\ 000\ \$$) : un calcul de **retour sur investissement** simple, à confirmer par une expérience.
- `ShelveLoc` (variable qualitative) : R crée des **indicatrices** par rapport à la modalité de **référence** `Bad`. Un bon emplacement en rayon est associé à 4.8 milliers d'unités de plus qu'un mauvais. Pour changer de référence : `relevel()`.

Le R^2 passe de 0.2 à 0.87.

4.5.2 Comparer des modèles

- Le R^2 augmente **toujours** quand on ajoute une variable : il ne permet pas de comparer des modèles de tailles différentes. On utilise le R^2 **ajusté**, l'**AIC** (plus petit = meilleur) ou, mieux, l'erreur de prédiction sur un jeu de **validation**.
- Deux modèles emboîtés se comparent avec un test de Fisher (`anova()`).

```
anova(modele_1, modele_2)
```

```
#> Analysis of Variance Table
#>
#> Model 1: Sales ~ Price
#> Model 2: Sales ~ Price + CompPrice + Advertising + ShelveLoc + Age + Income
#>   Res.Df  RSS Df Sum of Sq   F Pr(>F)
#> 1    398 2552
#> 2    392  407  6      2145 344 <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
AIC(modele_1, modele_2)
```

```
#>          df  AIC
#> modele_1  3 1882
#> modele_2  9 1160
```

4.5.3 Interactions

L'effet du prix dépend-il de l'emplacement en rayon ? Le terme `Price:ShelveLoc` (ou `Price * ShelveLoc`, qui inclut aussi les effets principaux) laisse la pente varier selon l'emplacement :

```
modele_3 <- lm(Sales ~ Price * ShelveLoc + CompPrice + Advertising + Age +
 ↪  Income,
               data = Carseats)
anova(modele_2, modele_3)
```

```
#> Analysis of Variance Table
#>
#> Model 1: Sales ~ Price + CompPrice + Advertising + ShelveLoc + Age + Income
#> Model 2: Sales ~ Price * ShelveLoc + CompPrice + Advertising + Age + Income
```

```
#>   Res.Df RSS Df Sum of Sq    F Pr(>F)
#> 1    392 407
#> 2    390 407  2     0.869 0.42   0.66
```

L'interaction n'est pas significative : l'effet du prix est le même quel que soit l'emplacement, ce que suggéraient les droites parallèles du graphique.

4.5.4 Évaluer la prédictibilité (cadre PCS)

On mesure la qualité prédictive sur des données **non utilisées** pour l'ajustement, et on la compare à une **référence** naïve (prédire la moyenne).

```
set.seed(2026)
indices_train <- sample(nrow(Carseats), size = round(0.7 * nrow(Carseats)))
train <- Carseats[indices_train, ]
test  <- Carseats[-indices_train, ]

rmse <- function(observe, predit) sqrt(mean((observe - predit)^2))

m1 <- lm(Sales ~ Price, data = train)
m2 <- lm(Sales ~ Price + CompPrice + Advertising + ShelveLoc + Age + Income,
 ↪  data = train)
c(reference = rmse(test$Sales, mean(train$Sales)),
  simple    = rmse(test$Sales, predict(m1, test)),
  multiple  = rmse(test$Sales, predict(m2, test)))
```

```
#> reference    simple  multiple
#>     2.971     2.638     1.059
```

La racine de l'erreur quadratique moyenne (RMSE) est en milliers d'unités. Le modèle multiple divise l'erreur par plus de deux. Changez la graine : le classement des modèles reste-t-il le même ? C'est une vérification de **stabilité**.

4.6 Régression logistique

4.6.1 Quand la réponse est binaire

Pour prédire une variable **binaire** (achète / n'achète pas, résilie / reste), la régression linéaire ne convient pas : elle peut prédire des valeurs hors de $[0, 1]$. La régression logistique modélise la **probabilité** $p = P(Y = 1 \mid X)$ via la fonction logit :

$$\log \frac{p}{1-p} = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p \quad \Longleftrightarrow \quad p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \cdots)}}$$

Le rapport $p/(1-p)$ est la **cote** (*odds*). Les coefficients sont estimés par **maximum de vraisemblance** avec `glm(..., family = binomial)`.

4.6.2 Exemple : choix de marque

Le jeu `OJ` (package `ISLR`) décrit 1 070 achats **réels** de jus d'orange dans cinq magasins américains, issus de données de caisse utilisées dans l'ouvrage *Business Analysis Using Regression* (Foster, Stine et Waterman, 1998). À chaque achat, le client a choisi entre deux marques, Citrus Hill (`CH`) et Minute Maid (`MM`). Qu'est-ce qui fait choisir Minute Maid ? Ces données sont réutilisées au chapitre 5.

Variable	Description
`Purchase`	marque achetée : `CH` ou `MM` (**la cible**)
`WeekofPurchase`	semaine de l'achat (numéro de semaine)
`StoreID`, `STORE`, `Store7`	magasin de l'achat (identifiant ; codage alternatif ; magasin n° 7 ou non)
`PriceCH`, `PriceMM`	prix de référence de chaque marque ($)
`DiscCH`, `DiscMM`	remise appliquée à chaque marque ($)
`SalePriceCH`, `SalePriceMM`	prix de vente, après remise ($)
`PctDiscCH`, `PctDiscMM`	remise en pourcentage du prix de référence
`PriceDiff`	prix de vente de MM moins prix de vente de CH ($)
`ListPriceDiff`	prix de référence de MM moins prix de référence de CH ($)
`SpecialCH`, `SpecialMM`	mise en avant spéciale de la marque en magasin (1 = oui)
`LoyalCH`	fidélité du client à CH, entre 0 et 1, calculée sur ses achats passés

```
oj <- OJ |> mutate(achat_mm = as.integer(Purchase == "MM"))
mean(oj$achat_mm)
```

```
#> [1] 0.3897
```

```
modele_log <- glm(achat_mm ~ PriceDiff + LoyalCH + SpecialMM + SpecialCH,
                  data = oj, family = binomial)
summary(modele_log)
```

```
#>
#> Call:
#> glm(formula = achat_mm ~ PriceDiff + LoyalCH + SpecialMM + SpecialCH,
#>     family = binomial, data = oj)
#>
#> Coefficients:
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)    3.241      0.237   13.66  < 2e-16 ***
#> PriceDiff     -2.771      0.376   -7.37  1.7e-13 ***
#> LoyalCH       -6.354      0.386  -16.44  < 2e-16 ***
#> SpecialMM      0.190      0.248    0.77     0.44
#> SpecialCH     -0.394      0.261   -1.51     0.13
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for binomial family taken to be 1)
#>
#>     Null deviance: 1430.85  on 1069  degrees of freedom
#> Residual deviance:  836.88  on 1065  degrees of freedom
#> AIC: 846.9
#>
#> Number of Fisher Scoring iterations: 5
```

Les coefficients sont sur l'échelle du **log de la cote**. Leur exponentielle est un **rapport de cotes** (*odds ratio*, OR) : OR > 1 augmente la cote d'acheter MM, OR < 1 la diminue.

```
round(exp(cbind(OR = coef(modele_log), confint.default(modele_log))), 3)
```

```
#>                OR  2.5 % 97.5 %
```

```
#> (Intercept) 25.548 16.050 40.668
#> PriceDiff    0.063  0.030  0.131
#> LoyalCH      0.002  0.001  0.004
#> SpecialMM    1.209  0.744  1.965
#> SpecialCH    0.674  0.405  1.124
```

- `PriceDiff` : quand MM devient plus cher que CH de 10 centimes, la cote d'acheter MM est multipliée par $e^{0{,}1\times\hat{\beta}} \approx 0.76$, soit une baisse d'environ 24 %.
- `LoyalCH` : la fidélité passée à CH est de loin le facteur le plus fort. En marketing, le comportement passé est souvent le meilleur prédicteur du comportement futur.
- Les mises en avant en magasin n'ont pas d'effet significatif ici.

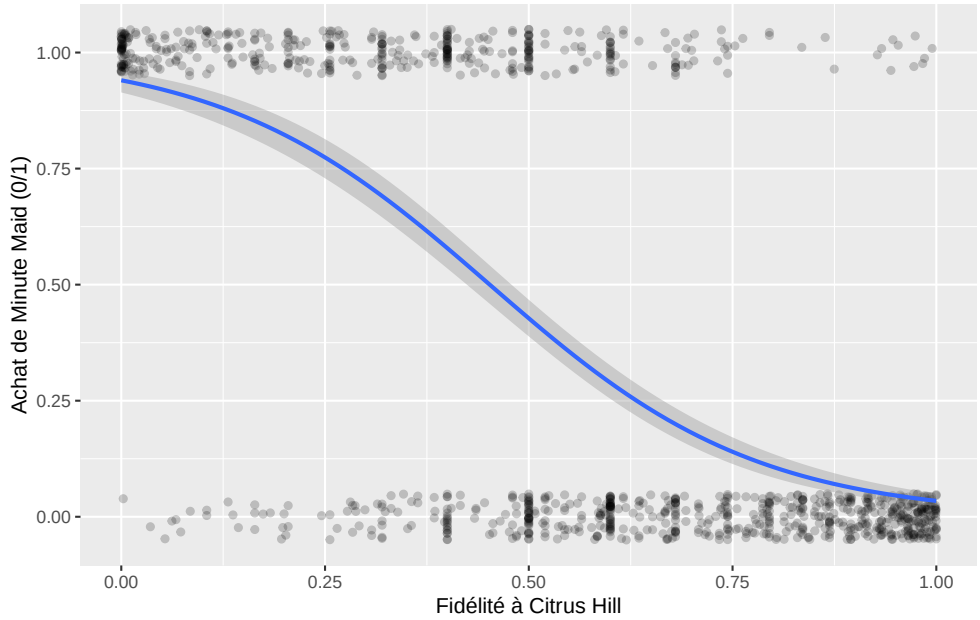
4.6.3 Prédire des probabilités

```
nouveaux <- tibble(PriceDiff = c(-0.2, 0.3), LoyalCH = 0.5, SpecialMM = 0,
 ↪  SpecialCH = 0)
predict(modele_log, newdata = nouveaux, type = "response")
```

```
#>      1      2
#> 0.6498 0.3170
```

Pour un client sans préférence (`LoyalCH = 0,5`), la probabilité d'acheter MM passe d'environ 65 % quand MM est 20 centimes moins cher à environ 32 % quand il est 30 centimes plus cher : une mesure de la **sensibilité au prix**.

```
ggplot(oj, aes(x = LoyalCH, y = achat_mm)) +
  geom_jitter(height = 0.05, alpha = 0.2) +
  geom_smooth(method = "glm", method.args = list(family = binomial), formula =
   ↪  y ~ x) +
  labs(x = "Fidélité à Citrus Hill", y = "Achat de Minute Maid (0/1)")
```



Transformer les probabilités en décisions (avec un **seuil**) et évaluer un classifieur (matrice de confusion, courbe ROC) est l'objet du chapitre 5.

4.7 Résumé

- Visualiser avant de modéliser (quartet d'Anscombe) ; un graphique a un message, des axes avec unités, une source.
- R de base : `plot()`, `hist()`, `boxplot()`, `barplot()` ; paramètres `col`, `pch`, `cex`, `main`, `xlab`… ; `par(mfrow = ...)`.
- `ggplot2` : `ggplot(données) + geom_*(aes(...))` ; mappages **dans** `aes()`, constantes **hors** de `aes()` ; `position` des barres ; `facet_wrap(vars())` ; `scale_*()` ; `labs()`, `theme_*()` ; `ggsave()`.
- Régression linéaire : `lm(y ~ x1 + x2, data)`, `summary()`, `coef()`, `confint()`, `predict()`, diagnostics avec `plot(modele)`.
- Interpréter chaque coefficient « toutes choses égales par ailleurs » ; oublier une variable importante biaise les autres ; association $\neq$ causalité.
- Comparer les modèles sur des données de **validation** (RMSE), face à une référence naïve.
- Régression logistique : `glm(y ~ ..., family = binomial)` ; `exp(coef())` donne les rapports de cotes ; `predict(type = "response")` les probabilités.

4.8 Exercices

Exercice 4.1 — `ggplot2` sur `rp2018`. On utilise le jeu `rp2018` (package `questionr`), qui contient des résultats du recensement de 2018 pour les communes de plus de 2 000 habitants, restreint à cinq départements :

```
data(rp2018)
rp <- filter(rp2018, departement %in% c("Oise", "Rhône", "Hauts-de-Seine",
                                        "Lozère", "Bouches-du-Rhône"))
```

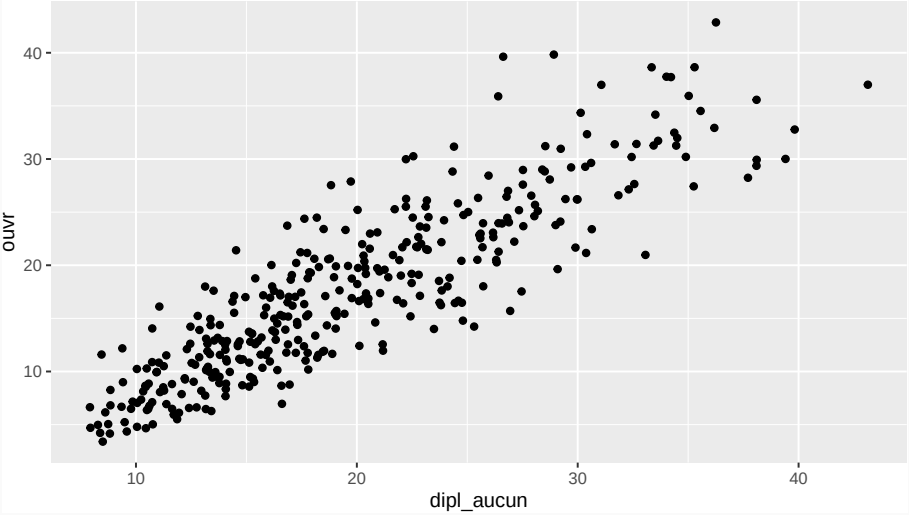
Variables utiles : `commune` (nom), `departement`, `pop_tot` (population totale), `pop_cl` (taille de la commune en classes), et des pourcentages : `dipl_aucun` (sans diplôme), `dipl_sup` (diplômés du supérieur), `ouvr` (ouvriers), `cadres`, `chom` (chômeurs), `proprio` (propriétaires).

1. Faire un nuage de points croisant le pourcentage de sans diplôme (`dipl_aucun`) et le pourcentage d'ouvriers (`ouvr`).
2. Refaire ce nuage avec des points rouges de transparence 0,2.
3. Représenter la répartition du pourcentage de propriétaires (`proprio`) selon la taille de la commune (`pop_cl`) sous forme de boîtes à moustaches.
4. Représenter le nombre de communes selon leur taille (`pop_cl`) sous forme de diagramme en bâtons.
5. Faire le nuage de points croisant le pourcentage de sans diplôme et le pourcentage d'ouvriers, en faisant varier la couleur selon le département.
6. Sur le même graphique, faire varier la taille des points selon la population totale (`pop_tot`).
7. Toujours sur le même graphique, rendre les points transparents (opacité 0,5).
8. Représenter la répartition du pourcentage de propriétaires selon la taille de la commune sous forme de boîtes à moustaches, en faisant varier la couleur de remplissage (`fill`) selon le département.
9. Représenter le nombre de communes selon leur taille sous forme de diagramme en bâtons empilés, avec une couleur différente par département.
10. Modifier le paramètre `position` pour afficher les barres les unes à côté des autres.
11. Modifier à nouveau `position` pour représenter, pour chaque taille de commune, les proportions de communes de chaque département.

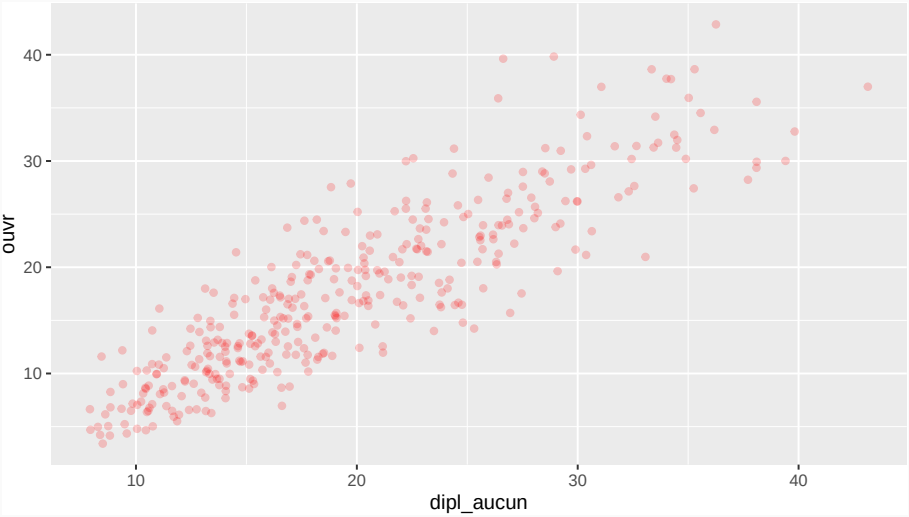
12. Faire un nuage de points du pourcentage de cadres (`cadres`, en abscisse) et du pourcentage de diplômés du supérieur (`dipl_sup`, en ordonnée), avec un graphique par département grâce à `facet_grid()`.
13. Sur le même graphique, faire varier la taille des points selon la population totale et rendre les points transparents.
14. Faire le nuage de points croisant le pourcentage de chômeurs (`chom`) et le pourcentage de sans diplôme, et y ajouter le nom des communes (`commune`), en rouge et en taille 2,5.
15. Dans le graphique précédent, n'afficher que le nom des communes ayant plus de 15 % de chômeurs.

Solution

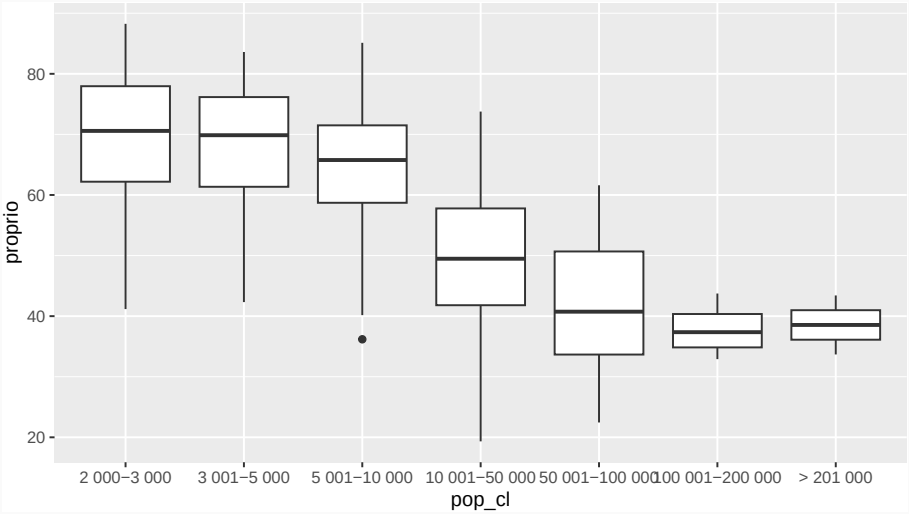
```
ggplot(rp) + geom_point(aes(x = dipl_aucun, y = ouvr))
 ↪  # 1
```



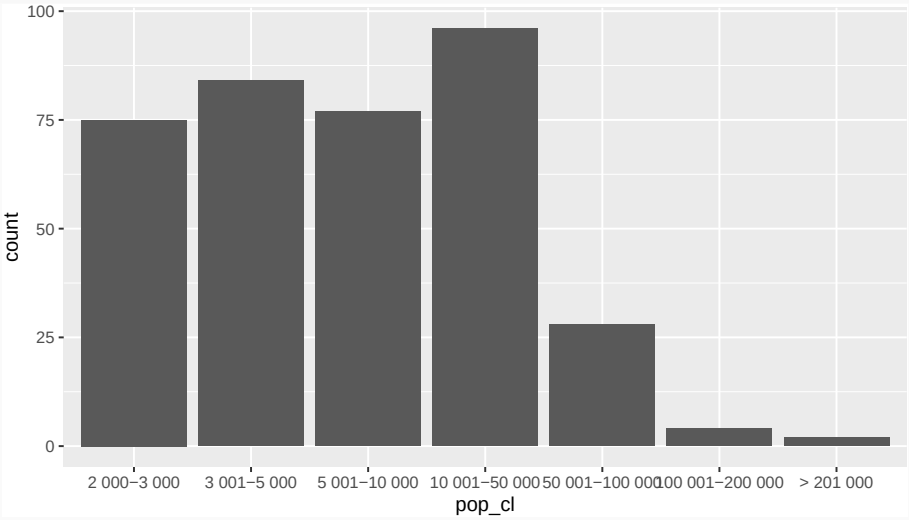

```
ggplot(rp) + geom_point(aes(x = dipl_aucun, y = ouvr), color = "red",
 ↪  alpha = 0.2)  # 2
```



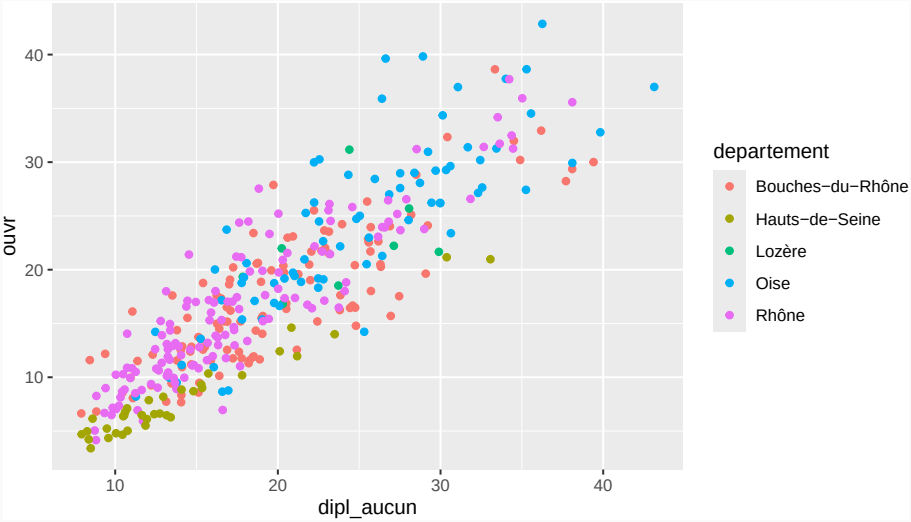

```
ggplot(rp) + geom_boxplot(aes(x = pop_cl, y = proprio))
 ↪  # 3
```



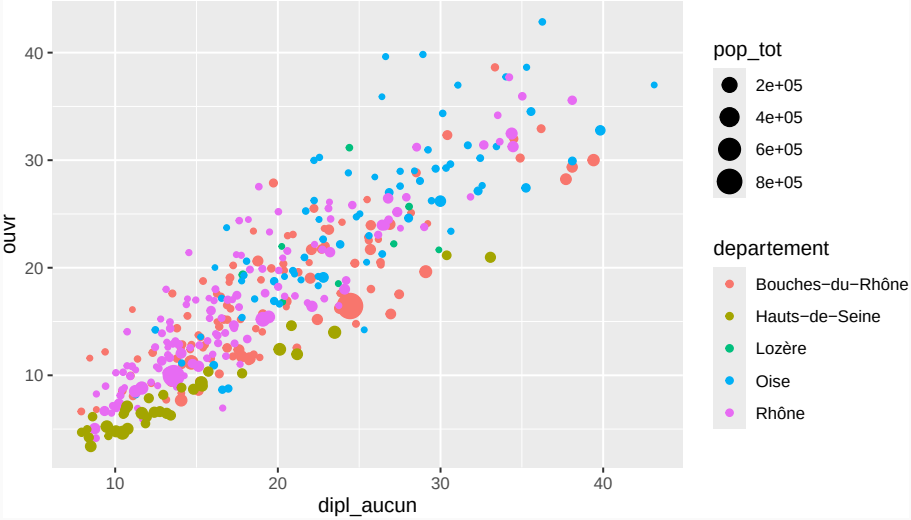

```
ggplot(rp) + geom_bar(aes(x = pop_cl))
 ↪  # 4
```



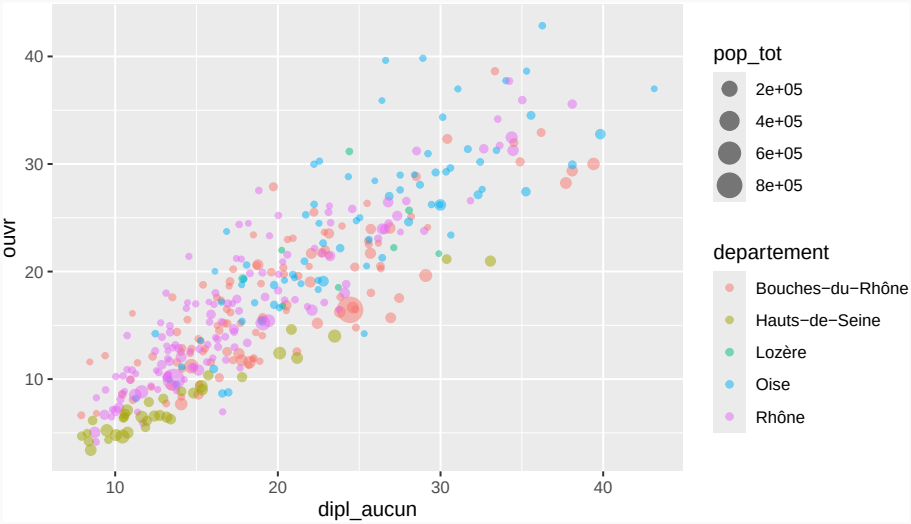

```
ggplot(rp) + geom_point(aes(x = dipl_aucun, y = ouvr, color =
 ↪  departement))    # 5
```



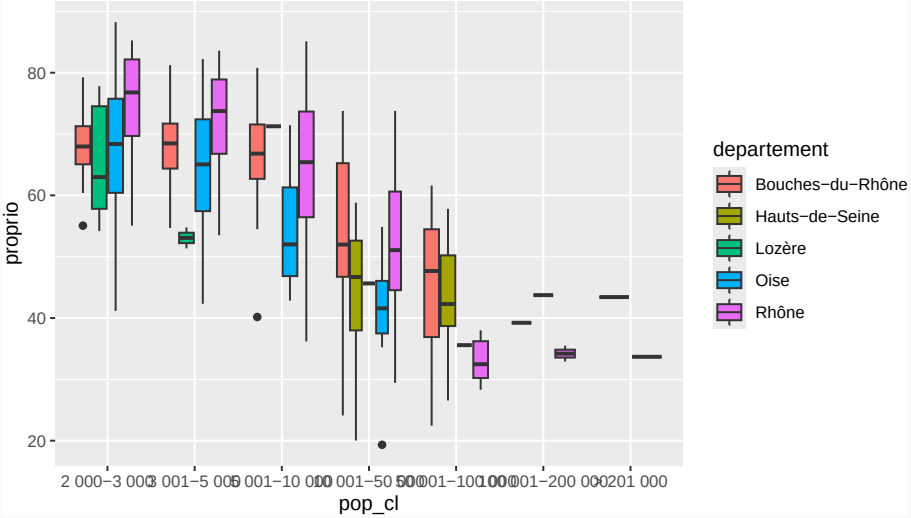
```
ggplot(rp) + geom_point(aes(x = dipl_aucun, y = ouvr, color = departement,
↪  # 6
                            size = pop_tot))
```



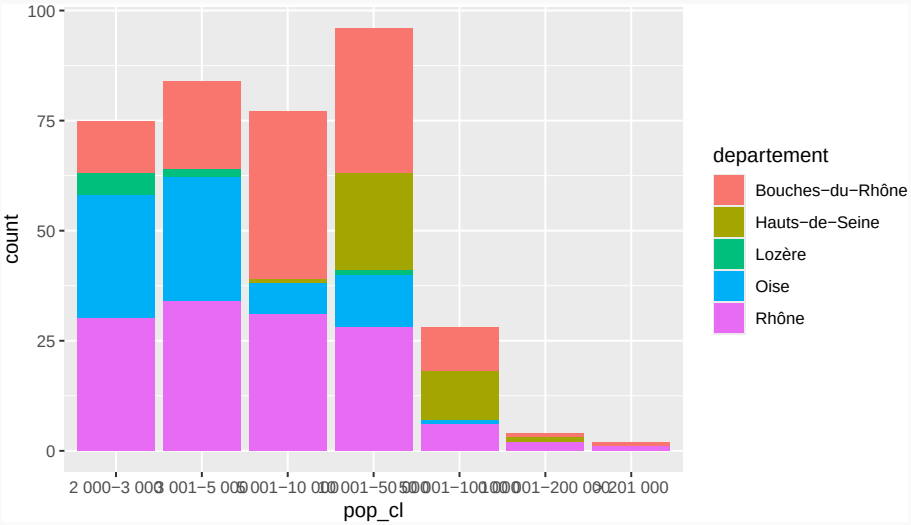

```
ggplot(rp) + geom_point(aes(x = dipl_aucun, y = ouvr, color = departement,
↪  # 7
                            size = pop_tot), alpha = 0.5)
```



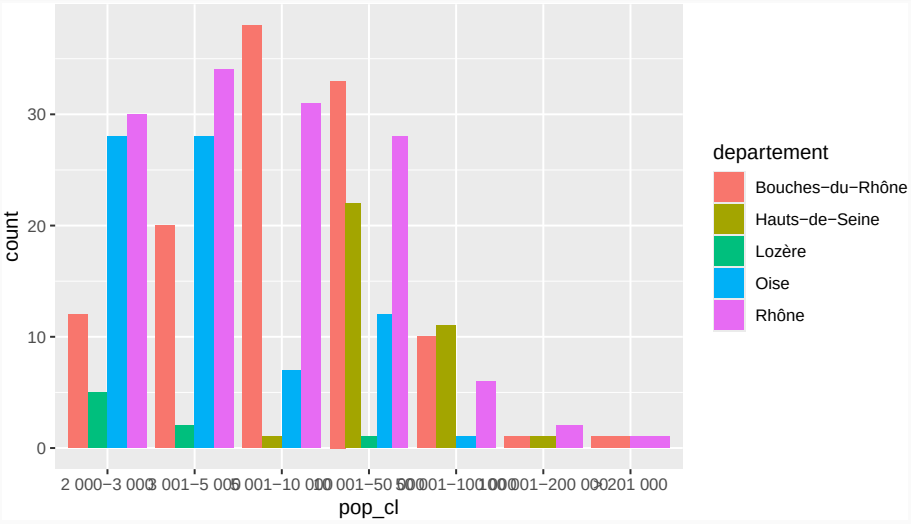

```
ggplot(rp) + geom_boxplot(aes(x = pop_cl, y = proprio, fill =
↪  departement))    # 8
```



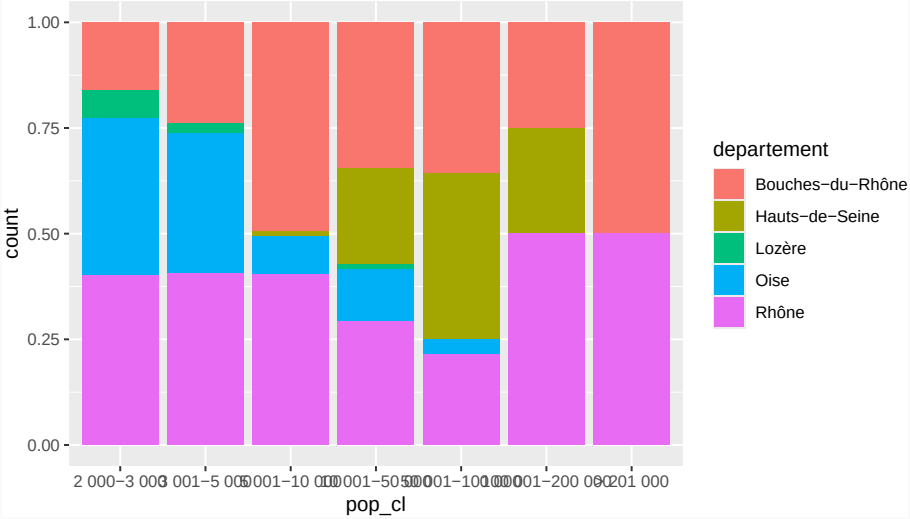

```
ggplot(rp) + geom_bar(aes(x = pop_cl, fill = departement))
↪  # 9
```



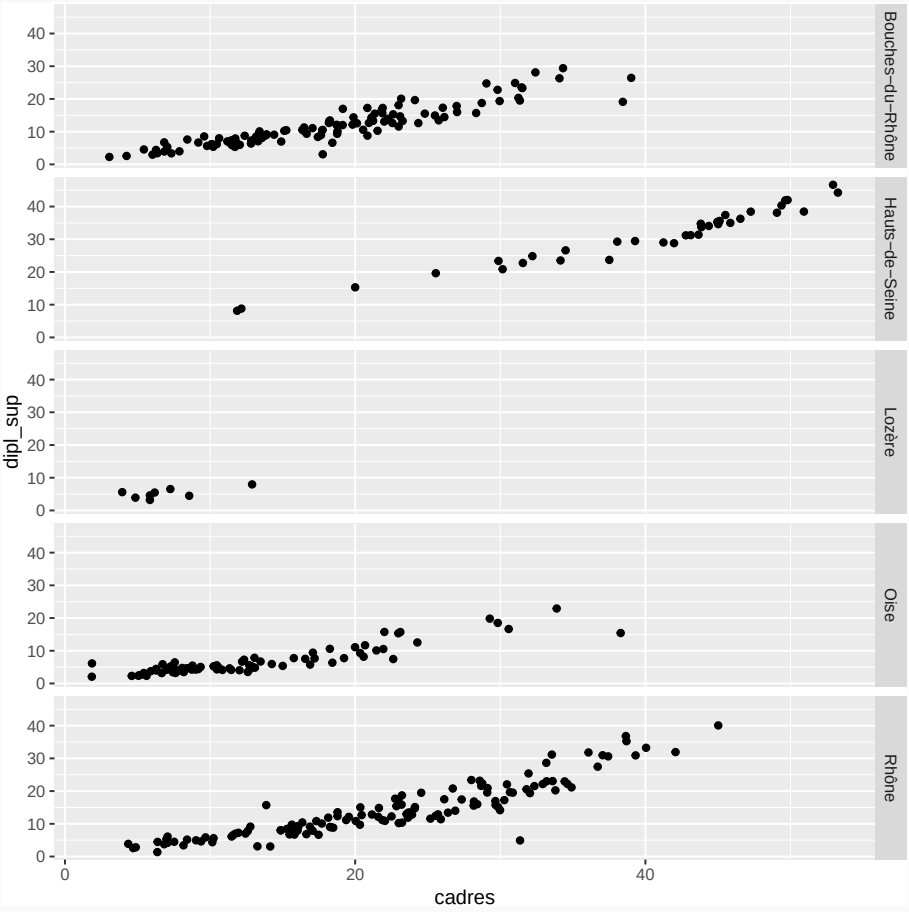

```
ggplot(rp) + geom_bar(aes(x = pop_cl, fill = departement), position =
↪  "dodge") # 10
```



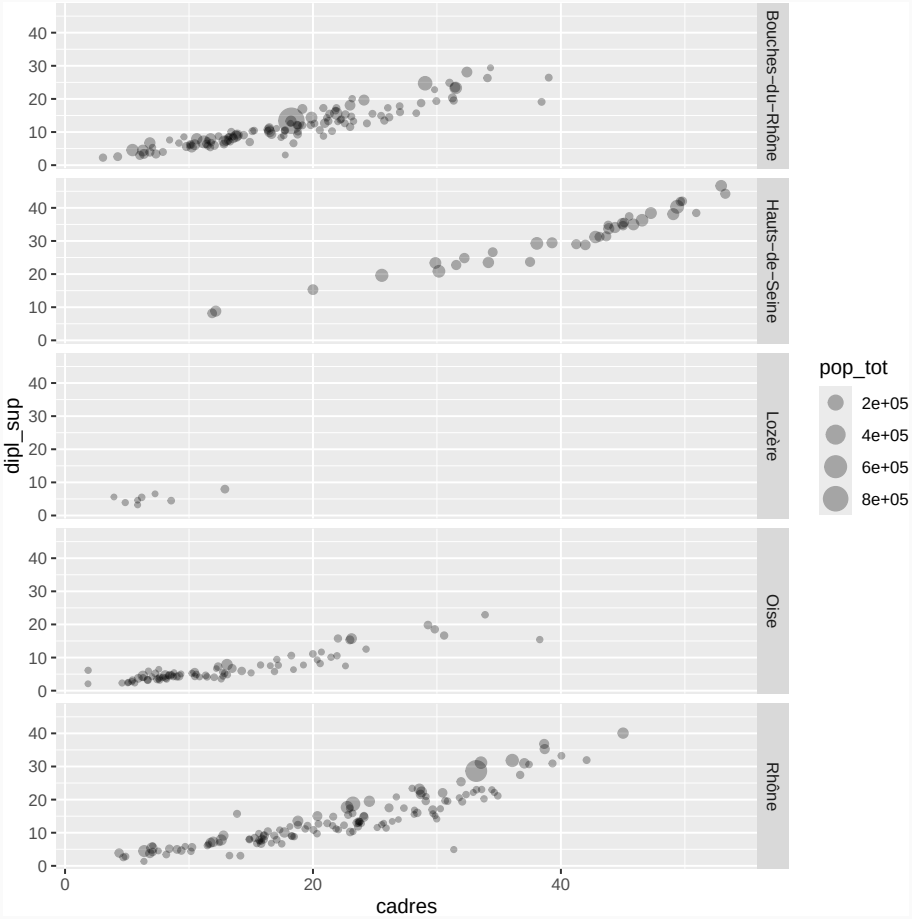
```
ggplot(rp) + geom_bar(aes(x = pop_cl, fill = departement), position =
 ↪  "fill")  # 11
```



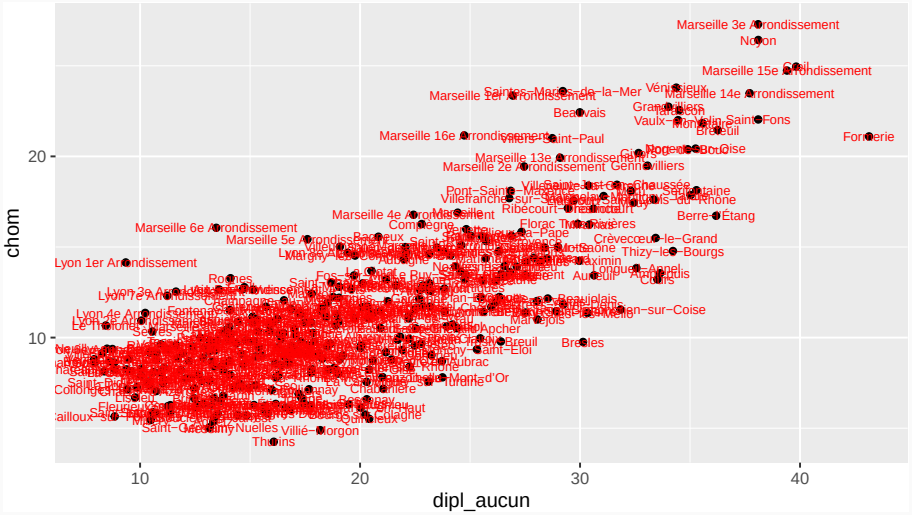

```
ggplot(rp) +
 ↪  # 12
  geom_point(aes(x = cadres, y = dipl_sup)) +
  facet_grid(rows = vars(departement))
```



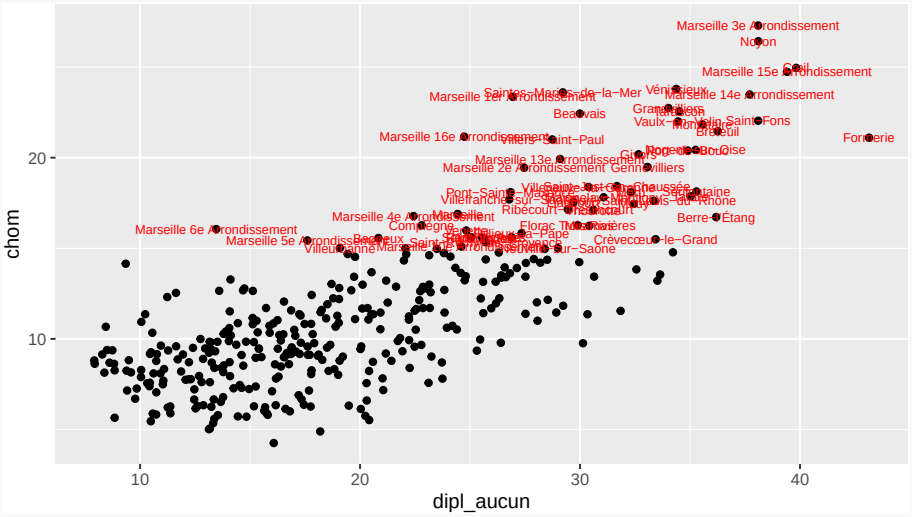
```
ggplot(rp) +
  # 13
  geom_point(aes(x = cadres, y = dipl_sup, size = pop_tot), alpha = 0.3) +
  facet_grid(rows = vars(departement))
```




```
ggplot(rp, aes(x = dipl_aucun, y = chom)) +
  # 14
  geom_point() +
  geom_text(aes(label = commune), color = "red", size = 2.5)
```



```r
ggplot(rp, aes(x = dipl_aucun, y = chom)) +
  # 15
  geom_point() +
  geom_text(data = filter(rp, chom > 15), aes(label = commune),
            color = "red", size = 2.5)
```



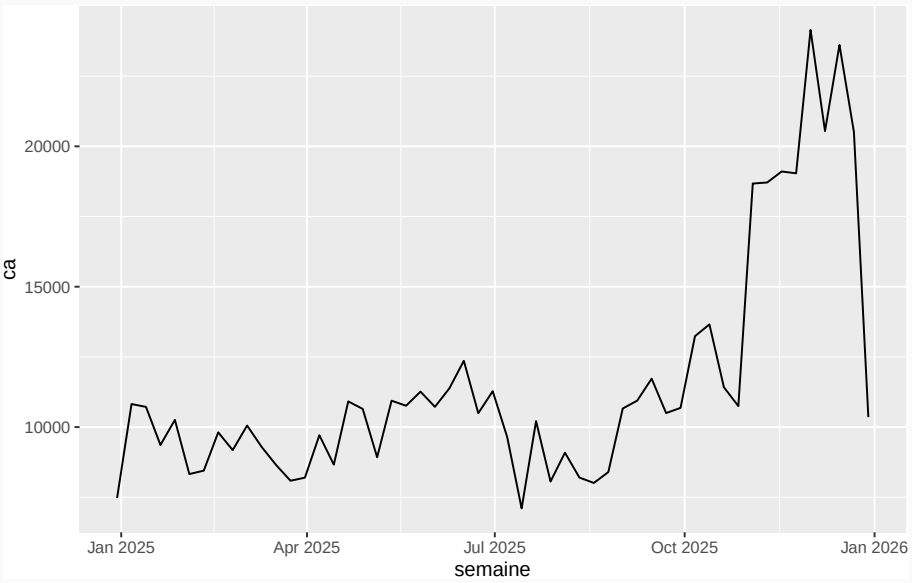
Au graphique 14, les étiquettes se superposent et deviennent illisibles : c'est pourquoi on n'affiche, en général, que les noms des points remarquables (graphique 15).

Exercice 4.2 — Tableau de bord de la boutique. Avec les tables de la boutique, produire :

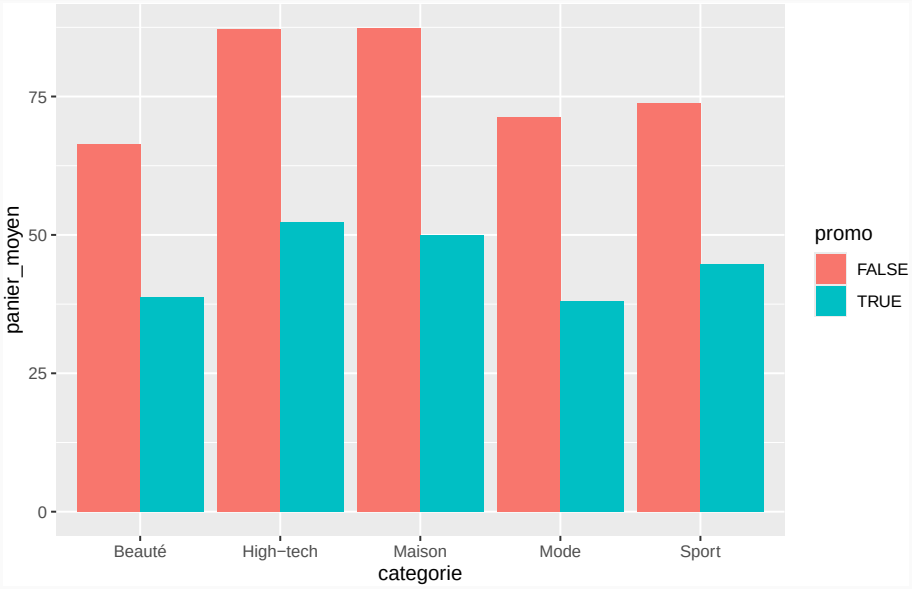
1. le chiffre d'affaires hebdomadaire de l'année (`floor_date(date, "week")`) ;
2. le panier moyen par catégorie et par statut promo, en barres côte à côte ;
3. la part des commandes en promotion par mois, en courbe ;
4. un graphique au choix, soigné (titre-message, axes, source), prêt à être présenté à la direction.

Solution (questions 1 à 3)

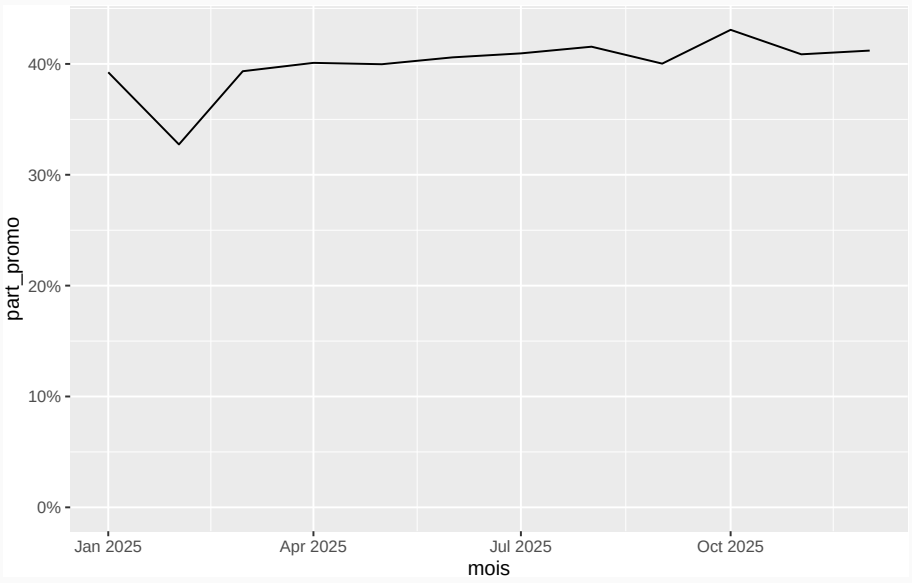
```r
commandes |>
  mutate(semaine = floor_date(date, "week", week_start = 1)) |>
  group_by(semaine) |>
  summarise(ca = sum(montant)) |>
  ggplot(aes(semaine, ca)) +
  geom_line()
```




```
commandes |>
  group_by(categorie, promo) |>
  summarise(panier_moyen = mean(montant), .groups = "drop") |>
  ggplot(aes(x = categorie, y = panier_moyen, fill = promo)) +
  geom_col(position = "dodge")
```




```
commandes |>
  mutate(mois = floor_date(date, "month")) |>
  group_by(mois) |>
  summarise(part_promo = mean(promo)) |>
  ggplot(aes(mois, part_promo)) +
  geom_line() +
  scale_y_continuous(labels = scales::percent, limits = c(0, NA))
```



Exercice 4.3 — Régression sur `Carseats`.

1. Ajuster `Sales ~ Advertising` puis ajouter `Price`, `CompPrice` et `ShelveLoc`. Comment évolue le coefficient de `Advertising` ?
2. Prédire les ventes d'un magasin avec `Price = 110`, `CompPrice = 125`, `ShelveLoc = "Medium"` et un budget publicitaire de 0, 10 puis 20 k$.
3. Les magasins hors des États-Unis (`US = "No"`) ont-ils un budget publicitaire différent ? Utiliser un graphique et un test.

Solution

```
coef(lm(Sales ~ Advertising, data = Carseats))
```

```
#> (Intercept) Advertising
#>      6.7370      0.1144
```

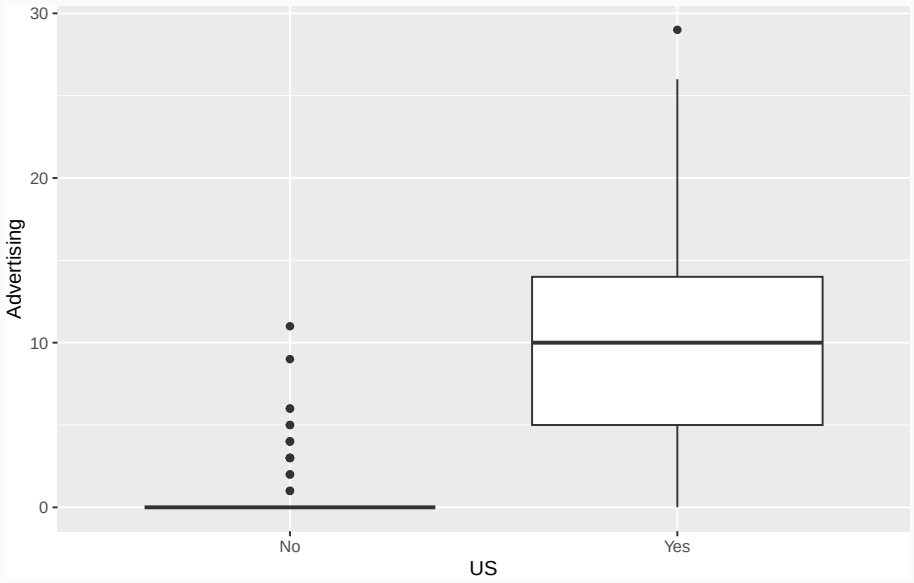
```
coef(lm(Sales ~ Advertising + Price + CompPrice + ShelveLoc, data =
 ↪  Carseats))
```

```
#>     (Intercept)     Advertising           Price       CompPrice
#>         3.84443         0.12028        -0.09359         0.09374
#>   ShelveLocGood ShelveLocMedium
#>         4.72932         1.77276
```

```
m <- lm(Sales ~ Advertising + Price + CompPrice + ShelveLoc, data =
 ↪  Carseats)
predict(m, newdata = tibble(Advertising = c(0, 10, 20), Price = 110,
                            CompPrice = 125, ShelveLoc = "Medium"))
```

```
#>     1     2     3
#> 7.040 8.243 9.446
```

```
ggplot(Carseats, aes(x = US, y = Advertising)) + geom_boxplot()
```




```
wilcox.test(Advertising ~ US, data = Carseats)
```

```
#>
#>  Wilcoxon rank sum test with continuity correction
#>
#> data:  Advertising by US
#> W = 2085, p-value <2e-16
#> alternative hypothesis: true location shift is not equal to 0
```

Hors des États-Unis, la grande majorité des magasins (125 sur 142) n'a aucun budget publicitaire.

Exercice 4.4 — Poids de naissance. Avec `phbirths` (`datasets/phbirths.dat`), modéliser le poids de naissance (`grams`) en fonction de la durée de gestation, puis ajouter le tabagisme de la mère (`smoke`). Interpréter le coefficient de `smoke`, puis ajuster une régression logistique du faible poids (moins de 2 500 g).

Solution

```
phbirths <- read.table("../datasets/phbirths.dat")
coef(lm(grams ~ gestate + smoke, data = phbirths))
```

```
#> (Intercept)     gestate   smokeTRUE
#>     -3027.0       162.0      -190.6
```

```
glm(I(grams < 2500) ~ gestate + smoke, data = phbirths, family = binomial)
 ↪  |>
  coef() |> exp() |> round(3)
```

```
#> (Intercept)     gestate   smokeTRUE
#>   1.061e+12   4.490e-01   2.349e+00
```

À durée de gestation égale, les bébés de mères fumeuses pèsent environ 190 g de moins en moyenne.

Pour aller plus loin

- J. Barnier, *Introduction à R et au tidyverse*, chapitre « Visualiser avec ggplot2 ».
- H. Wickham, *ggplot2: Elegant Graphics for Data Analysis*, 3e éd. (ggplot2-book.org).
- G. James et al., *An Introduction to Statistical Learning*, chapitres 3 (régression linéaire) et

4 (classification), en accès libre ; les jeux `Carseats` et `OJ` en sont issus.